



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

РАСЧЁТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ
НА ТЕМУ:
«Анализ свойств строковых образцов,
содержащих повторные переменные»

Студент _____
(Группа)

(Подпись, дата)

(И. О. Фамилия)

Руководитель ВКР

(Подпись, дата)

(И. О. Фамилия)

Консультант

(Подпись, дата)

(И. О. Фамилия)

Нормоконтролёр

(Подпись, дата)

(И. О. Фамилия)

2026 г.

АННОТАЦИЯ

Записка состоит из 62 страниц, содержит 4 разделов, 13 листингов, 6 таблиц и 5 рисунков. Работа посвящена анализу однозначности строковых шаблонов с повторяющимися переменными. Разработаны обёртка-анализатор с базой уже известных шаблонов и эвристики быстрой оценки однозначности и неоднозначности шаблона (балансирующие разрезы, унарные и бинарные критерии, перенос результатов по морфизмам), не обращающиеся к решателю. Для перехода к уравнениям в словах предложено и обосновано ускоренное отсечение ветвей нильсеновского разбиения на основе классов коммутации переменных, реализованное как расширение SMT-решателя Ostrich.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Исследовательская часть	6
1.1 Основные обозначения	6
1.2 Задача определения однозначности шаблона	6
1.3 Избыточные образцы: критерий Рейденбаха	8
1.4 Монотонность однозначности относительно морфизмов	9
1.5 Балансирующие позиции и критерий неоднозначности	10
1.6 Критерий однозначности по балансирующему разрезу	12
1.7 Уравнения в словах	13
1.8 Сложность задачи выполнимости уравнений в словах	14
1.9 SMT-решатели и теория строк	15
1.10 Решатель Z3	15
1.11 Решатель Ostrich	17
1.12 Нильсеновские преобразования и умные разбиения	18
1.13 Интерполяция Крейга в строковых теориях	19
1.14 Проблемы при применении решателей к задаче однозначности	20
1.15 Периодичность слов: теорема Файна–Вилфа и коммутативность	21
2 Конструкторская часть	24
2.1 Функциональные требования	24
2.2 Общая архитектура (каркас)	25
2.3 Проектирование интерфейса	26
2.4 Проектирование базы образцов	26
2.5 Каноническая форма образца и ключ базы	27
2.6 Обзор направлений проектирования	28
2.7 Эвристика однозначности	29
2.8 Эвристики неоднозначности	30
2.9 Перенос результатов по морфизмам	31
2.10 Спецификация SMT-задачи и выбор формы	31

2.11	Общий алгоритм с классами коммутации	34
2.12	Сводный управляющий алгоритм	36
3	Технологическая часть	38
3.1	Стек технологий и внешние библиотеки	38
3.2	Обвязка над решателем Ostrich	39
3.3	Хранение, обход и конвертация шаблонов	39
3.4	Внутренние структуры Ostrich и вариант с классами ком- мутации	40
3.5	Логирование и визуализация дерева вывода	42
3.6	Описание программы и руководство пользователя	42
4	Аналитическая часть	45
4.1	Тестирование программного обеспечения и анализ резуль- татов	45
4.1.1	Верификация и оценка эвристического алгоритма	45
4.1.2	Оценка производительности специализированного решателя Ostrich	46
4.1.3	Эффективность применения эвристики коммутативности	48
4.1.4	Анализ имеющейся базы данных и синтез методов	49
	ЗАКЛЮЧЕНИЕ	52
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	53
	ПРИЛОЖЕНИЕ А	55

ВВЕДЕНИЕ

Задача проверки выполнимости уравнений в словах (word equations) встречается во многих разделах информатики: в статическом анализе программ, верификации протоколов безопасности и символьном исполнении кода. Уравнение в словах — это равенство двух слов, составленных из литералов и переменных; решить его значит найти подстановку строк вместо переменных, при которой равенство выполняется. На практике такие уравнения обрабатывают SMT-решателями с поддержкой теории строк; из распространённых решателей этого класса можно назвать Z3 [1] и Ostrich [2].

У однозначных образцов есть и отдельное прикладное применение. А. Ливчаком было отмечено, что однозначный образец позволяет записать систему уравнений в словах одним уравнением: если подставить термы вместо переменных однозначного образца, конъюнкция покомпонентных равенств сворачивается в одно строковое уравнение. Например, для однозначного образца $x A y x B y$ система из двух уравнений $E_1 = F_1$, $E_2 = F_2$ равносильна одному уравнению $E_1 A E_2 E_1 B E_2 = F_1 A F_2 F_1 B F_2$. Таким образом, однозначный образец служит конструктором систем уравнений в экзистенциальной теории строк, и распознавание однозначности образцов имеет практический смысл.

Сама задача определения однозначности шаблона (pattern ambiguity) ставится так: по заданному образцу нужно установить, существует ли строка, соответствующая ему более чем одним способом. Эта задача сводится к системе строковых ограничений, которую можно передать SMT-решателю. Скорость решения при этом во многом определяется эвристиками, управляющими разбиением уравнений. Решатель Ostrich использует эвристику нильсеновского разбиения, однако на уравнениях с удвоенными нетерминалами (шаблон $x^J \cdot y \cdot \Phi_1 = y^I \cdot x \cdot \Phi_2$) его базовая реализация может порождать потенциально бесконечное дерево вывода. Это и делает нужной отдельную оптимизацию.

Целью работы является анализ однозначности строковых шаблонов с повторяющимися переменными. Для её достижения решаются следующие задачи: разработать обёртку-анализатор с базой уже разобранных шаблонов; разработать эвристики быстрой оценки однозначности и неоднозначности шаблона вне решателя; исправить и ускорить отсечение ветвей при переходе к уравнениям в словах

в решателе Ostrich. Однозначность — довольно редкое свойство шаблонов: в частности, оно требует, чтобы каждая переменная входила в шаблон не менее двух раз. Поэтому штатных эвристик решателей, рассчитанных в основном на уравнения, близкие к линейным, для этой задачи не хватает.

Научная новизна работы состоит в следующем. Во-первых, критерий морфической примитивности Рейденбаха перенесён на образцы с константами: введено понятие балансирующей позиции (линейная зависимость векторов кратностей частей), и на его основе сформулированы критерии неоднозначности — унарный, критерий плавных границ и бинарный. Во-вторых, на основе леммы о коммутировании при повторении (опирающейся на теорему Файна–Вилфа) в решатель Ostrich добавлена оптимизация на основе классов коммутации переменных, которая убирает бесконечное ветвление нильсеновского разбиения на структуре удвоения. В-четвёртых, перечисленные методы собраны в единый конвейер (обёртка с накопительной базой образцов, каскад эвристик и доработанный решатель); его работа проверена экспериментально.

Методологическую основу работы составляют комбинаторика на словах (теория периодичности, теорема Файна–Вилфа, морфизмы слов), методы теории SMT и теории строк (процедура $DPLL(T)$, нильсеновские преобразования, интерполяция Крейга), а также экспериментальное сравнение решателей на выборке образцов.

Объект исследования — строковые шаблоны с повторяющимися переменными и порождаемые ими уравнения в словах; предмет — свойство однозначности таких шаблонов и методы (эвристические и на основе решателей) его определения. Практическая значимость работы в том, что разработанные эвристики и доработанный решатель позволяют распознавать однозначность более широкого класса шаблонов за приемлемое время. Помимо анализа программ, однозначные шаблоны применяются в кодировании и биоинформатике, где их рассматривают как кортежи с запасом устойчивости к ошибке.

1 Исследовательская часть

1.1 Основные обозначения

Пусть Σ — конечный алфавит, Σ^* — множество всех конечных слов над Σ , включая пустое слово ε . Множество Σ^* образует моноид относительно операции конкатенации. Пусть X — счётное множество переменных, $X \cap \Sigma = \emptyset$. Множество слов над расширенным алфавитом $\Sigma \cup X$ обозначается $(\Sigma \cup X)^*$; буквы алфавита называются *константами* (литералами), элементы X — переменными.

Подстановка $\sigma : X \rightarrow \Sigma^*$ продолжается до гомоморфизма $\hat{\sigma} : (\Sigma \cup X)^* \rightarrow \Sigma^*$ стандартным образом (тождественно на константах и согласованно с конкатенацией). Через $|w|$ обозначается длина слова w , а через $|w|_x$ — число вхождений переменной x в слово w .

В терминах этих обозначений ниже формулируется центральная задача работы — определение однозначности шаблона, — а её сведение к *уравнениям в словах* (подраздел 1.7) служит мостом к алгоритмическим методам решения.

1.2 Задача определения однозначности шаблона

Пусть Σ — алфавит, $\mathcal{P}$ — шаблон (pattern) над Σ , то есть слово над расширенным алфавитом $\Sigma \cup X$, где X — переменные. Говорят, что шаблон $\mathcal{P}$ *неоднозначен* (ambiguous), если существует строка $w \in \Sigma^*$, допускающая два различных соответствия шаблону:

$$\exists \sigma_1, \sigma_2 : X \rightarrow \Sigma^*, \quad \sigma_1 \neq \sigma_2, \quad \hat{\sigma}_1(\mathcal{P}) = \hat{\sigma}_2(\mathcal{P}) = w. \quad (1)$$

Задача определения однозначности сводится к проверке выполнимости уравнения:

$$\mathcal{P}[\sigma_1] = \mathcal{P}[\sigma_2] \wedge \sigma_1 \neq \sigma_2, \quad (2)$$

что при раскрытии ограничений $\sigma_1 \neq \sigma_2$ через длины или позиции переменных даёт систему строковых ограничений в теории строк.

Для шаблонов, содержащих повторяющиеся переменные, характерно наличие нелинейных уравнений, где одна переменная встречается более одного раза в одной и той же части уравнения. Например, для шаблона $x \cdot y \cdot x$ уравнение

однозначности принимает вид:

$$x_1 \cdot y_1 \cdot x_1 = x_2 \cdot y_2 \cdot x_2, \quad (3)$$

что является нелинейным уравнением в словах. Именно такие уравнения наиболее эффективно обрабатываются нильсеновскими методами.

Однозначность образца имеет и самостоятельное прикладное значение: она позволяет кодировать систему уравнений в словах одним уравнением. Следующее утверждение (лемма Ливчака) использует однозначность образца $xAuxBy$ как *склеивающий конструктор*.

Лемма 1 (Ливчак: склейка уравнений однозначным образцом). *Пусть $A, B \in \Sigma$ — различные константы. Тогда для любых строковых термов E_1, E_2, F_1, F_2 система из двух уравнений в словах*

$$E_1 = F_1, \quad E_2 = F_2 \quad (4)$$

равносильна (при любой интерпретации входящих переменных) единственному уравнению

$$E_1 A E_2 E_1 B E_2 = F_1 A F_2 F_1 B F_2. \quad (5)$$

Доказательство. Левая и правая части уравнения (5) являются образами образца $xAuxBy$ при подстановках $(x, y) \mapsto (E_1, E_2)$ и $(x, y) \mapsto (F_1, F_2)$. По лемме 5 образец $xAuxBy$ однозначен, поэтому равенство образов равносильно совпадению подстановок, то есть $E_1 = F_1$ и $E_2 = F_2$. $\square$

Более общо, всякий однозначный образец задаёт в экзистенциальной теории строк *конструктор систем уравнений*: подстановка термов вместо его переменных кодирует конъюнкцию покомпонентных равенств одним строковым уравнением, не расширяя выразительной силы теории. Это служит дополнительной мотивацией исследования — распознавание однозначных образцов даёт компактные кодировки систем строковых ограничений.

1.3 Избыточные образцы: критерий Рейденбаха

Для шаблона $\mathcal{P}$ над $\Sigma \cup X$, $X = \{x_1, \dots, x_k\}$, и его фрагмента (подслова) f введём *вектор кратностей переменных*

$$\nu(f) = (|f|_{x_1}, |f|_{x_2}, \dots, |f|_{x_k}) \in \mathbb{N}^k, \quad (6)$$

где $|f|_{x_j}$ — число вхождений переменной x_j в f (вхождения констант не учитываются).

Д. Рейденбах и Ю. Шнайдер [3] ввели понятие *морфической примитивности* слов, формализующее структурную избыточность шаблона.

Определение 1. *Шаблон $\mathcal{P}$ называется морфически непримитивным, или избыточным (prolix), если существуют более короткое слово $\mathcal{Q}$, $|\mathcal{Q}| < |\mathcal{P}|$, и морфизмы g, h такие, что $h(\mathcal{Q}) = \mathcal{P}$ и $g(\mathcal{P}) = \mathcal{Q}$. В противном случае $\mathcal{P}$ морфически примитивен, или лаконичен (succinct). Эквивалентно: шаблон морфически примитивен тогда и только тогда, когда он не является неподвижной точкой никакого нетривиального морфизма.*

Пример 1. *Шаблон $x_1x_2x_1$ морфически примитивен. Шаблон $x_1x_2x_1x_2 = (x_1x_2)^2$ избыточен: он является неподвижной точкой нетривиального морфизма φ с $\varphi(x_1) = x_1x_2$, $\varphi(x_2) = \varepsilon$, что отвечает сжатию $g(x_1) = x_1, g(x_2) = \varepsilon$ к слову $\mathcal{Q} = x_1x_1$ и обратному разворачиванию $h(x_1) = x_1x_2$.*

Связь избыточности с неоднозначностью устанавливает следующий результат.

Теорема 1 (Фрайденбергер, Рейденбах, Шнайдер [4]). *Бестерминальный шаблон допускает однозначный нестирающий морфный образ тогда и только тогда, когда он морфически примитивен. В частности, всякий избыточный (prolix) шаблон неоднозначен: каждый его нестирающий морфизм неоднозначен, а потому существуют две различные подстановки с одинаковым образом.*

Таким образом, морфическая непримитивность даёт *достаточное структурное условие неоднозначности*: обнаружив избыточность образца, обёртка решателя помечает его как неоднозначный, не запуская дорогостоящий перебор. Ограничение критерия Рейденбаха — он сформулирован для бестерминальных

образцов (без констант) и опирается на морфическую факторизацию по отдельным позициям. Ниже это условие расширяется на образцы с константами.

1.4 Монотонность однозначности относительно морфизмов

Свойства однозначности и неоднозначности шаблона ведут себя монотонно относительно двух классов морфизмов — по переменным и по буквам алфавита. Это позволяет переносить уже установленный результат с одного шаблона на целые семейства родственных.

Определение 2. Морфизм $\varphi : (\Sigma \cup X)^* \rightarrow (\Sigma \cup X)^*$ называется *переменным*, если он тождествен на алфавите ($\varphi(a) = a$ для всех $a \in \Sigma$) и отображает каждую переменную в слово над $\Sigma \cup X$. Морфизм $\psi : \Sigma^* \rightarrow \Sigma'^*$, продолженный тождественно на X , называется *буквенным*.

Лемма 2 (Монотонность неоднозначности). Пусть шаблон $\mathcal{P}$ неоднозначен. Тогда:

1. (прообраз по переменным) для всякого шаблона $\mathcal{Q}$ и переменного морфизма φ с $\varphi(\mathcal{Q}) = \mathcal{P}$ шаблон $\mathcal{Q}$ также неоднозначен;
2. (образ по алфавиту) для всякого буквенного морфизма ψ , не отождествляющего свидетелей неоднозначности (в частности, для любого инъективного кодирования), шаблон $\psi(\mathcal{P})$ неоднозначен.

Доказательство. Пусть $\sigma_1 \neq \sigma_2$ — свидетели неоднозначности $\mathcal{P}$, то есть $\hat{\sigma}_1(\mathcal{P}) = \hat{\sigma}_2(\mathcal{P}) = w$.

(1) Положим $\tau_i = \hat{\sigma}_i \circ \varphi$, то есть $\tau_i(z) = \hat{\sigma}_i(\varphi(z))$ для $z \in X$. Тогда $\hat{\tau}_i(\mathcal{Q}) = \hat{\sigma}_i(\varphi(\mathcal{Q})) = \hat{\sigma}_i(\mathcal{P}) = w$, то есть τ_1, τ_2 дают одно слово. Подстановки различны: σ_1 и σ_2 различаются на некоторой переменной t , входящей в $\mathcal{P} = \varphi(\mathcal{Q})$; значит, t входит в $\varphi(z)$ для некоторой $z \in \text{vars}(\mathcal{Q})$, откуда $\hat{\tau}_1(z) \neq \hat{\tau}_2(z)$. Следовательно, $\mathcal{Q}$ неоднозначен.

(2) Положим $\tau_i = \psi \circ \sigma_i$. Тогда $\hat{\tau}_i(\psi(\mathcal{P})) = \psi(\hat{\sigma}_i(\mathcal{P})) = \psi(w)$, а так как ψ не отождествляет σ_1 и σ_2 , подстановки τ_1, τ_2 различны. Следовательно, $\psi(\mathcal{P})$ неоднозначен. $\square$

Контрапозиция леммы 2 даёт двойственное утверждение для однозначных шаблонов.

Следствие 1 (Монотонность однозначности). *Пусть шаблон $\mathcal{P}$ однозначен. Тогда:*

1. *(образ по переменным) для всякого переменного морфизма φ шаблон $\varphi(\mathcal{P})$ однозначен;*
2. *(прообраз по алфавиту) для всякого буквенного морфизма ψ (не отождествляющего свидетелей) и шаблона $\mathcal{R}$ с $\psi(\mathcal{R}) = \mathcal{P}$ шаблон $\mathcal{R}$ однозначен.*

Пример 2. *Из неоднозначности шаблона $xAyBux$ немедленно следует:*

- *неоднозначность $xAz yBuzzx$ (переменный прообраз: при $z \mapsto \varepsilon$ он переходит в $xAyBux$);*
- *неоднозначность $xAyAyx$ (буквенный образ при $B \mapsto A$).*

Обратно, из однозначности шаблона $xAyxBy$ следует однозначность $xuAyxuBy$ (переменный образ при $x \mapsto xu$). Случаи с повторяющимися константами (как $xAyAyx$) в используемом наборе тестов не встречаются, однако критерий монотонности полезно учитывать при пополнении базы шаблонов.

1.5 Балансирующие позиции и критерий неоднозначности

Определение 3. *Позиция, разбивающая образец $\mathcal{P} = w_1w_2$, называется балансирующей, если оба вектора кратностей частей ненулевые и линейно зависимы (пропорциональны): $\nu(w_1) = c\nu(w_2)$ для некоторого $c \in \mathbb{Q}_{>0}$. Требование ненулевости существенно: если бы одна из частей не содержала переменных, её образ имел бы строго постоянную длину, и разрез не нёс бы информации о соотношении длин переменных, на котором основаны последующие критерии.*

Пример 3. *В образце $xxAyBxux$ балансирующей является позиция между y и B : $\nu(xxAy) = (2,1) = \nu(Bxux)$. В образце $xuAyxBy$ балансирующих позиций две — на буквах A и B : $\nu(xu) = (1,1) \parallel \nu(yxBxu) = (2,2)$ и $\nu(xuAyx) = (2,2) \parallel \nu(xu) = (1,1)$.*

Лемма 3 (Унарный критерий неоднозначности). Пусть балансирующая позиция образца P приходится на букву M , причём слева и справа от неё встречаются только переменные и одна другая буква $c \neq M$. Тогда при $k \geq 2$ образец P неоднозначен, и свидетелей неоднозначности можно выбрать унарными словами над $\{c\}$.

Доказательство. Подставим каждую переменную x_j унарным словом c^{a_j} , $a_j \geq 0$. Так как кроме переменных образец содержит лишь единственное вхождение M (балансирующая позиция) и буквы c , его образ имеет вид $c^{B(a)} M c^{A(a)}$, где $B(a) = \kappa_1 + \nu(w_1) \cdot a$ и $A(a) = \kappa_2 + \nu(w_2) \cdot a$ — числа букв c слева и справа от M , $a = (a_1, \dots, a_k)$, а κ_1, κ_2 — постоянные вклады констант. Два присваивания a и a' дают один и тот же образ тогда и только тогда, когда $\nu(w_1) \cdot (a - a') = 0$ и $\nu(w_2) \cdot (a - a') = 0$. В силу балансирующей позиции $\nu(w_1) = c \nu(w_2)$, поэтому обе системы сводятся к одному уравнению $\nu(w_2) \cdot \delta = 0$, $\delta = a - a'$. При $k \geq 2$ оно имеет ненулевое целочисленное решение δ ; выбрав a достаточно большим и $a' = a + \delta \geq 0$, получаем две различные подстановки с общим образом. $\square$

Пример 4. Для $xuAyxVxu$ балансирующая позиция на A окружена переменными и буквой V ; унарные свидетели над $\{V\}$: $\sigma_1 = (x=V, y=\varepsilon)$ и $\sigma_2 = (x=\varepsilon, y=V)$ дают общий образ $VABVV$. Балансирующая позиция на V окружена переменными и буквой A ; унарные свидетели над $\{A\}$: $\sigma_1 = (x=A, y=AA)$ и $\sigma_2 = (x=AA, y=A)$ дают общий образ A^7VA^3 .

Если унарный критерий неприменим, неоднозначность можно установить через плавность границ между фрагментами.

Определение 4. Граница между фрагментами u и v называется плавной относительно периодического слова w^∞ , если из того, что u и v — подслова w^∞ , следует, что uv — подслово w^∞ .

Лемма 4 (Критерий бесконечной неоднозначности). Если все границы между фрагментами образца, полученными разбиением по всем балансирующим позициям, плавны относительно некоторого периодического слова w^∞ , то образец бесконечно неоднозначен. Для слова $w = c_1c_2$ плавность проверяется покомпонентно:

— граница вида xx (либо yu) плавна $\iff$ переменная является степенью c_1c_2 или степенью c_2c_1 ;

- граница вида xA плавна $\iff$ либо $c_1 = A$ и $x = (c_2)^? (Ac_2)^i$, либо $c_2 = A$ и $x = (A)^? (c_1A)^i c_1$ (аналогично для прочих констант и для границ, где константа стоит первой);
 - граница вида xu плавна $\iff x = (c_1)^? (c_2c_1)^i$ и $y = c_2 (c_1c_2)^j (c_1)^?$,
- где множитель, помеченный $(\cdot)^?$, необязателен.

Следствие 2. Критерий 4 обобщает критерий избыточности Рейденбаха (теорема 1). Если в качестве границ рассматривать не отдельные позиции, одиночные буквы и пустые (нулевые) границы, а граничные слова над попарно непересекающимися (*disjoint*) алфавитами переменных, не входящими в разделяемые этими словами линейно зависимые фрагменты, то морфически непримитивные (*prolix*) образцы становятся частным случаем бесконечно неоднозначных по лемме 4.

1.6 Критерий однозначности по балансирующему разрезу

Балансирующие позиции дают и достаточное условие однозначности, двойственное критерию неоднозначности.

Лемма 5 (Однозначность по балансирующему разрезу). Пусть образец $\mathcal{P}$ допускает балансирующий разрез $\mathcal{P} = w_1w_2$, при котором

$$w_1 = x_i C_1 w'_1, \quad w_2 = x_i C_2 w'_2, \quad C_1 \neq C_2, \quad (7)$$

где $C_1, C_2 \in \Sigma$ — различные константы, а x_i — одна и та же переменная в начале обоих фрагментов. Тогда $\mathcal{P}$ однозначен по переменной x_i . Симметрично, такой же разрез в суффиксах ($w_1 = w'_1 C_1 x_i$, $w_2 = w'_2 C_2 x_i$) обеспечивает однозначность по x_i . В случае двух переменных однозначность по x_i влечёт однозначность образца в целом.

Доказательство. Пусть σ_1, σ_2 — подстановки с общим образом w , $\hat{\sigma}_1(\mathcal{P}) = \hat{\sigma}_2(\mathcal{P}) = w$. Из балансирующего разреза $\nu(w_1) = c \nu(w_2)$ следует, что длины образов фрагментов выражаются через одну и ту же сумму $S = \sum_j \nu(w_2)_j |\sigma(x_j)|$ как $|\hat{\sigma}(w_1)| = cS + d_1$ и $|\hat{\sigma}(w_2)| = S + d_2$, где d_1, d_2 — суммарные длины констант

в w_1, w_2 . Поскольку полная длина $|w| = (c+1)S + d_1 + d_2$ фиксирована, величина S , а с ней и точка разреза $|\hat{\sigma}(w_1)|$, определяются одинаково для σ_1 и σ_2 . Значит, образ разбивается на $w = w^{(1)}w^{(2)}$ в одной и той же позиции, причём $\hat{\sigma}_1(w_1) = \hat{\sigma}_2(w_1) = w^{(1)}$ и $\hat{\sigma}_1(w_2) = \hat{\sigma}_2(w_2) = w^{(2)}$ — фиксированные слова.

Оба фрагмента w_1, w_2 начинаются с одной переменной x_i , поэтому при любой подстановке σ слова $w^{(1)} = \sigma(x_i)C_1 \cdots$ и $w^{(2)} = \sigma(x_i)C_2 \cdots$ имеют общий префикс $\sigma(x_i)$, сразу за которым стоят *различные* константы $C_1 \neq C_2$. Поэтому первая позиция, в которой фиксированные слова $w^{(1)}$ и $w^{(2)}$ различаются, равна $|\sigma(x_i)| + 1$. Так как эта позиция не зависит от выбора σ , получаем $|\sigma_1(x_i)| = |\sigma_2(x_i)|$, а вместе с совпадением общего префикса — $\sigma_1(x_i) = \sigma_2(x_i)$. Следовательно, образец однозначен по x_i . $\square$

Пример 5. Шаблон $xAuxBy$ допускает балансирующий разрез $xAy \mid xBy$ с $\nu(xAy) = (1,1) = \nu(xBy)$, где $x_i = x$, $C_1 = A \neq B = C_2$; по лемме 5 он однозначен. Аналогично шаблон $xAuuxxBy$ допускает разрез $xAuux \mid xBy$ с $\nu(xAuux) = (2,2) \parallel \nu(xBy) = (1,1)$ и теми же $x_i = x$, $C_1 = A$, $C_2 = B$, поэтому также однозначен.

1.7 Уравнения в словах

Структурные критерии предыдущих подразделов разрешают задачу однозначности лишь для образцов специального вида. В общем случае проверка однозначности сводится к проверке выполнимости *уравнения в словах*, что и переводит задачу на язык алгоритмических методов.

Определение 5. Уравнение в словах над алфавитом Σ и множеством переменных X — это пара $(u, v) \in (\Sigma \cup X)^* \times (\Sigma \cup X)^*$, записываемая как $u = v$.

Подстановка σ называется *решением* уравнения $u = v$, если $\hat{\sigma}(u) = \hat{\sigma}(v)$. Уравнение в словах *выполнимо*, если у него существует хотя бы одно решение; конечный набор уравнений образует *систему*, для которой решением служит подстановка, удовлетворяющая всем уравнениям одновременно.

Пример уравнения в словах: $x \cdot a = a \cdot y$, где $a, b \in \Sigma$ — литералы, $x, y \in X$ — переменные. Одним из его решений является $x = a \cdot b$, $y = b \cdot a$ (тогда обе части равны aba); решением является и $x = y = a^k$ при любом $k \geq 0$.

Как показано выше, задача определения однозначности шаблона $\mathcal{P}$ сводится к выполнимости уравнения $\mathcal{P}[\sigma_1] = \mathcal{P}[\sigma_2]$, в котором каждая переменная встречается ровно дважды (по одному разу с каждой стороны равенства). Поэтому далее рассматриваются именно такие уравнения — с повторяющимися переменными.

Задача разрешимости уравнений в словах была поставлена А. Марковым в 1954 году и оставалась открытой до работы Маканина [5], который доказал, что она разрешима. Впоследствии Пландовски [6] установил, что задача лежит в классе PSPACE (подробнее — в подразделе о сложности). Задача разрешимости системы уравнений в словах также является PSPACE-полной.

1.8 Сложность задачи выполнимости уравнений в словах

Теоретическое изучение сложности задачи выполнимости уравнений в словах прошло несколько важных этапов, определивших современное понимание её трудоёмкости.

Первый фундаментальный результат принадлежит Г. С. Маканину [5], который в 1977 году доказал разрешимость задачи выполнимости одного уравнения в словах. Алгоритм Маканина основан на методе *обобщённых уравнений* (generalized equations): произвольное уравнение кодируется в специальный формат, в котором явно указаны позиционные перекрытия переменных. Доказывается, что если уравнение выполнимо, то решение может быть найдено за конечное число применений фиксированного набора разрешающих преобразований. Несмотря на то что исходный алгоритм Маканина имел неэлементарную сложность, сам факт разрешимости поставил задачу в разряд алгоритмически разрешимых.

В 2004 году В. Пландовски [6] существенно улучшил этот результат, доказав принадлежность задачи классу PSPACE.

Теорема 2 (Пландовски, 2004). *Задача выполнимости уравнения в словах над конечным алфавитом является PSPACE-полной.*

Аналогичный результат справедлив для систем уравнений в словах: добавление нескольких уравнений не меняет класса сложности — системы также PSPACE-полны.

Данные теоретические ограничения накладывают принципиальный предел на эффективность алгоритмов: полиномиального алгоритма для решения произвольных уравнений в словах (при предположении $P \neq PSPACE$) не существует. На практике, однако, задачи с небольшим числом переменных и структурированных ограничений решаются существенно быстрее. Это мотивирует разработку специализированных эвристик для конкретных классов уравнений.

Линейные уравнения решаются за линейное время методом последовательного применения подстановок. Нелинейные уравнения формально PSPACE-трудны, однако их структура допускает применение специализированных алгоритмов. Для задачи определения однозначности шаблона с n переменными преобразуется в уравнение для проверки на неоднозначность : каждая из n переменных встречается в нём ровно два раза (по одному разу с каждой стороны равенства). Этот класс уравнений составляет основной предмет исследования данной работы.

1.9 SMT-решатели и теория строк

SMT-решатели (Satisfiability Modulo Theories) — это системы автоматического доказательства теорем, позволяющие проверять выполнимость формул первого порядка относительно заданной теории. Стандарт SMT-LIB 2.6 [7] определяет единый интерфейс для работы с такими решателями, включая теорию строк `QF_S` с операциями конкатенации (`str.++`), длины (`str.len`), вхождения подстроки (`str.contains`), взятия подстроки (`str.substr`) и другими.

Формула теории строк в SMT-LIB 2.6 представляет собой булеву комбинацию строковых ограничений вида:

$$\varphi ::= (= t_1 t_2) \mid (\text{str.contains } t_1 t_2) \mid (\leq (\text{str.len } t) n) \mid \dots, \quad (8)$$

где t_1, t_2 — строковые термы, n — арифметическое выражение.

1.10 Решатель Z3

Z3 [1] — универсальный SMT-решатель общего назначения, разработанный в Microsoft Research Леонардо де Моурой и Николасом Бйёрнером. Решатель поддерживает широкий спектр теорий: линейную и нелинейную арифметику над

целыми и рациональными числами, теорию массивов, теорию битовых векторов, теорию неинтерпретированных функций, а также теорию строк. Z3 применяется в системах верификации программ (Dafny, Boogie), инструментах символьного исполнения, средствах проверки моделей и статического анализа программного обеспечения.

Архитектура $DPLL(T)$. В основе Z3 лежит алгоритм $DPLL(T)$ (Davis–Putnam–Logemann–Loveland modulo Theories) — расширение классического алгоритма DPLL для работы с теориями. Задача выполнимости формулы первого порядка разбивается на пропозициональную и теоретическую составляющие:

1. *Пропозициональный решатель* (SAT-ядро): атомарные ограничения заменяются булевыми переменными, формируя пропозициональный скелет. Ядро находит пропозиционально выполнимое присваивание или доказывает его отсутствие, используя механизм Conflict-Driven Clause Learning (CDCL).
2. *Теоретический оракул* (Т-решатель): при нахождении пропозиционально выполнимого присваивания набор соответствующих ограничений теории передаётся Т-решателю, который проверяет их совместность. При обнаружении конфликта генерируется *лемма конфликта* — дизъюнкция отрицаний конфликтного подмножества ограничений.
3. *Цикл взаимодействия*: лемма конфликта добавляется в пропозициональную формулу, исключая данное присваивание. Процесс повторяется до нахождения выполнимого присваивания, согласованного с теорией, или доказательства невыполнимости.

Для комбинирования нескольких теорий Z3 применяет *процедуру Нельсона–Оппена*: Т-решатели попеременно обмениваются равенствами между общими переменными, добиваясь согласованной совместной модели.

Подход Z3 к уравнениям в словах. Для чистых уравнений в словах вида $u = v$, где $u, v \in (\Sigma \cup X)^*$, Z3 применяет комбинированный подход: упрощение правилами переписывания, извлечение ограничений на длины $\text{len}(u) = \text{len}(v)$ для арифметического Т-решателя, позиционный анализ соотношений между переменными и ветвление в рамках $DPLL(T)$, оформленное как добавление дизъюнктивных лемм в пропозициональный решатель.

Ограничения Z3 на нелинейных уравнениях. Несмотря на универсальность, Z3 демонстрирует ряд ограничений при обработке нелинейных уравнений с повторяющимися переменными:

- Автоматный подход, эффективный для регулярных ограничений, плохо масштабируется на уравнениях с большим числом переменных: пересечение автоматов требует числа состояний, экспоненциального от числа операндов.
- Длиновые эвристики Z3 не учитывают алгебраическую структуру уравнений — в частности, факт повторения переменной. Для уравнения $x \cdot x = y \cdot z$ Z3 не использует информацию о том, что длина x должна быть кратна сумме длин y и z , делённой на два.
- Для задач однозначности шаблонов с повторяющимися переменными Z3 в ряде случаев не завершает работу за разумное время, тогда как специализированные решатели, использующие нильсеновские преобразования, справляются с данным классом задач.

Таким образом, Z3 является мощным универсальным инструментом, ориентированным на широкий спектр задач, однако не специализированным для нелинейных уравнений в словах. Это обосновывает актуальность специализированных решателей и оптимизированных эвристик нильсеновского разбиения, рассматриваемых в данной работе.

1.11 Решатель Ostrich

Ostrich [2] — SMT-решатель для строковых ограничений, разработанный в рамках фреймворка Princess [8]. Решатель реализован на языке Scala и использует процедуру Нельсона–Оппена для комбинирования теорий. Основной алгоритм решения строковых ограничений состоит из следующих компонентов:

1. *Предварительная обработка*: строковые ограничения, включая регулярные выражения и операции подстроки, преобразуются в систему уравнений-конкатенаций вида $\text{str} \cdot ++(x_1, x_2) = y$.
2. *Нильсеновское разбиение*: процедура NielsenSplitter применяет правила нильсеновского преобразования, строя дерево доказательства в фреймворке Princess.

3. *Проверка ограничений в модели длин*: для каждого листа дерева доказательства проверяется совместность ограничений на длины с помощью линейного программирования.
4. *Проверка выполнимости*: если ограничения в модели длин совместны, строится конкретное решение.

Фреймворк Princess предоставляет механизм плагинов через интерфейс `Plugin.Action`. Основные типы действий:

- `Plugin.AxiomSplit` — создаёт разбиение: текущая цель делится на две подцели, каждая из которых содержит добавленную аксиому и список действий по удалению фактов;
- `Plugin.AddAxiom` — добавляет аксиому в текущую цель без создания дополнительных ветвей;
- `Plugin.RemoveFacts` — удаляет указанные факты из текущей цели.

1.12 Нильсеновские преобразования и умные разбиения

Ядром строкового поиска в Ostrich служит метод Нильсена [9] — последовательное применение элементарных преобразований, уменьшающих длину решения уравнения (само уравнение при этом может и удлиниться). Рассмотрим уравнение $u = v$, где $u, v \in (\Sigma \cup X)^*$. Если оба слова непусты, возможен один из следующих случаев для первых символов u и v :

1. первые символы совпадают — они сокращаются, и задача сводится к более короткому уравнению;
2. первые символы — различные литералы — уравнение немедленно противоречиво, ветвь отбрасывается;
3. первый символ u — литерал a , первый символ v — переменная x — тогда либо $x = \varepsilon$ и литерал a сопоставляется с дальнейшим содержимым v , либо x начинается с a , то есть $x = a \cdot x'$, и производится замена $x \mapsto a \cdot x'$;
4. первый символ u — переменная x , первый символ v — переменная y — тогда либо x является префиксом y ($x = x'$, $y = x \cdot y'$), либо y является префиксом x ($y = y'$, $x = y \cdot x'$).

Случай «литерал против переменной» также порождает ветвление: либо переменная пуста ($x = \varepsilon$), либо начинается с данного литерала. Однако ветвь с пустой подстановкой сокращает число переменных, поэтому вдоль неё не может возникнуть бесконечной цепочки шагов.

Случай «переменная против переменной» порождает ветвление, при котором обе ветви вводят новую переменную, не уменьшая их общего числа. Именно это ветвление является источником экспоненциального роста числа шагов в худшем случае.

В формальной записи нильсеновское разбиение при появлении переменных x и y в начале слов u и v приводит к двум подзадачам:

$$x = y \cdot r \Rightarrow u[x \mapsto y \cdot r] = v[x \mapsto y \cdot r], \quad (9)$$

$$y = x \cdot r \Rightarrow u[y \mapsto x \cdot r] = v[y \mapsto x \cdot r], \quad (10)$$

где r — свежая переменная. Таким образом, метод строит, возможно бесконечное, дерево разбора, листьями которого являются либо противоречия, либо выполнимые тривиальные уравнения.

Эффективность метода критически зависит от *выбора точки разбиения*. Наивный выбор порождает избыточные ветви, тогда как *умные* (структурно-осведомлённые) разбиения выбирают точку с учётом дополнительной информации — модели ограничений на длины и алгебраической структуры уравнения (в частности, повторных вхождений переменных). Именно качество эвристики выбора разбиения определяет, завершится ли поиск за приемлемое число шагов; разработка таких эвристик для уравнений, порождаемых задачами однозначности шаблона, составляет предмет настоящей работы.

1.13 Интерполяция Крейга в строковых теориях

Задача однозначности шаблона кодируется *позитивной* теорией: набором уравнений в словах, в каждом из которых присутствует хотя бы одна явная константа, вместе с ограничениями на длины переменных (точная форма SMT-спецификации рассматривается в конструкторской части). Для такой теории естественно поставить вопрос, связанный с интерполяцией Крейга: можно ли ограничить алфавит модели лишь теми буквами, которые явно встречаются в теории.

Лемма 6 (Достаточность явного алфавита). *Пусть теория состоит только из уравнений в словах, в каждом из которых встречается хотя бы одна явная константа, и из ограничений на длины. Тогда у всякой её модели можно заменить все буквы, не входящие в теорию явно, на одну из явных констант, и полученная подстановка снова будет моделью.*

Доказательство. Пусть $\Sigma_0 \subseteq \Sigma$ — множество констант, явно входящих в теорию, и $c \in \Sigma_0$. Рассмотрим буквенный морфизм $h : \Sigma^* \rightarrow \Sigma_0^*$, тождественный на Σ_0 и отображающий каждую «постороннюю» букву в c . Морфизм сохраняет конкатенацию, поэтому из $\hat{\sigma}(u) = \hat{\sigma}(v)$ следует $h(\hat{\sigma}(u)) = h(\hat{\sigma}(v))$, то есть все уравнения остаются выполненными для подстановки $h \circ \sigma$. Так как h не стирающий ($|h(w)| = |w|$), ограничения на длины также сохраняются. Следовательно, $h \circ \sigma$ — модель теории. $\square$

Таким образом, для позитивной теории без регулярных ограничений при поиске решения достаточно рассматривать лишь алфавит явных констант уравнения — это и есть строковый аналог свойства интерполяции Крейга (общий «словарь» двух частей формулы достаточен для построения модели).

Важно, что при добавлении *регулярных ограничений* вида `str.in_re` это свойство перестаёт выполняться: `regex` фиксирует допустимый алфавит значений переменных, и заменять буквы уже нельзя. Более того, регулярные ограничения нельзя безболезненно отбросить, так как они влияют на стратегию разбиения уравнений в `Ostrich`. При выборе точки разбиения `Ostrich` сначала строит модель ограничений на длины и выбирает разбиение по этой модели, а регулярные ограничения могут взаимодействовать с длиновыми условиями и тем самым менять выбор точки; используемое правило разбиения по существу соответствует описанному в [10]. Это рассуждение обосновывает выбор SMT-спецификации (позитивная форма с явной константой и регулярными ограничениями по алфавиту), детально рассматриваемый в конструкторской части.

1.14 Проблемы при применении решателей к задаче однозначности

Существующая реализация нильсеновского разбиения в `Ostrich` использует стандартное ветвление при встрече двух переменных в начале (или конце) обеих

частей уравнения. При анализе задач однозначности шаблона наблюдается специфическая структура уравнений: одна переменная встречается дважды в одной части уравнения («удвоенная» переменная), тогда как в другой части стоят два различных символа.

Например, уравнение $x \cdot y \cdot r_1 = y \cdot x \cdot r_2$ имеет слева удвоенную переменную x , а справа — переменные y и x . В существующей реализации при стандартном нильсеновском разбиении для такого уравнения порождается ветвление, не использующее информацию об удвоении.

Ключевое наблюдение состоит в том, что при наличии двух нетерминалов x и y , встречающихся в обеих частях уравнения (или частично покрывающих друг друга), можно ввести понятие *класса коммутативности* и использовать его для нормализации уравнения, избегая тем самым ненужных ветвлений. Такой подход естественно назвать *умным* (структурно-осведомлённым) разбиением: вместо слепого перебора точек сплита решатель распознаёт структуру удвоения (11) и по лемме о коммутировании при удвоении (лемма 7, доказываемая в следующем подразделе 1.15) сразу выводит коммутирование переменных, превращая потенциально бесконечную ветвь в одно детерминированное упрощение. Теоретическое обоснование этого приёма — периодичность и коммутативность слов — вынесено в заключительный подраздел исследовательской части как надстройка над постановкой задачи.

1.15 Периодичность слов: теорема Файна–Вилфа и коммутативность

Этот заключительный подраздел даёт теоретическую надстройку над выявленной выше проблемой удвоения переменных: основной приём устранения бесконечного ветвления — вывод коммутирования $xy = yx$ — обосновывается фактами о периодичности слов. В отличие от предыдущих подразделов, относящихся к структуре самих образцов, изложенные здесь утверждения касаются *уравнений в словах*, возникающих при разбиении, и потому вынесены отдельно. Опираются они на классические факты комбинаторики на словах, связывающие повторяемость подслов с периодичностью [11].

Определение 6. *Натуральное число p , $1 \leq p \leq |w|$, называется периодом слова $w = w_1w_2 \cdots w_n$ (где $w_i \in \Sigma$), если $w_i = w_{i+p}$ для всех $1 \leq i \leq n - p$. Слово z называется корнем слова w , если $w = z^k$ для некоторого $k \geq 1$.*

Теорема 3 (Файн, Вилф [12]). *Если слово w обладает периодами p и q и его длина удовлетворяет неравенству $|w| \geq p + q - \gcd(p, q)$, то w обладает также периодом $\gcd(p, q)$. Граница $p + q - \gcd(p, q)$ неулучшаема.*

Теорема Файна–Вилфа влечёт фундаментальный критерий коммутативности слов, известный также как теорема Линдона–Шютценберже [13].

Следствие 3 (Критерий коммутативности). *Для любых слов $u, v \in \Sigma^*$ следующие условия эквивалентны:*

1. $uv = vu$;
2. существуют слово $z \in \Sigma^*$ и числа $k, m \geq 0$ такие, что $u = z^k$ и $v = z^m$;
3. u и v являются степенями одного общего корня.

Доказательство. Импликация (2) $\Rightarrow$ (1) очевидна: $z^k z^m = z^{k+m} = z^m z^k$. Докажем (1) $\Rightarrow$ (2), считая $u, v \neq \varepsilon$ (иначе утверждение тривиально). Рассмотрим периодическое слово $p = (uv)^\infty = uvuvuv \cdots$. Поскольку $uv = vu$, сдвиг p влево на $|u|$ позиций даёт $vuuv \cdots = (vu)^\infty = (uv)^\infty = p$; следовательно, $|u|$ — период p . Аналогично, записывая $p = (vu)^\infty$ и сдвигая на $|v|$, получаем, что $|v|$ — также период p . Возьмём префикс W слова p длины $|u| + |v|$: он обладает периодами $|u|$ и $|v|$, а так как $|W| = |u| + |v| \geq |u| + |v| - \gcd(|u|, |v|)$, по теореме 3 слово W обладает периодом $d = \gcd(|u|, |v|)$. Пусть z — префикс W длины d . Поскольку u и v являются префиксами W длины, кратной d , получаем $u = z^{|u|/d}$ и $v = z^{|v|/d}$. $\square$

Для задачи однозначности шаблонов критерий коммутативности применяется в следующей специализированной форме, описывающей структуру уравнений с удвоенными переменными, возникающих при нильсеновском разбиении (подраздел 1.12).

Лемма 7 (Коммутирование при удвоении). *Пусть переменные $x, y \in X$, контексты $\Phi_1, \Phi_2 \in (\Sigma \cup X)^*$, а подстановка σ является решением уравнения*

$$x^J y \Phi_1 = y^I x \Phi_2, \quad I, J \geq 1. \quad (11)$$

Тогда значения $\sigma(x)$ и $\sigma(y)$ коммутируют: $\sigma(x)\sigma(y) = \sigma(y)\sigma(x)$, и по следствию 3 являются степенями общего корня.

Доказательство. Обозначим $a = \sigma(x)$, $b = \sigma(y)$. Равенство сторон (11) означает, что одно и то же слово w при $I, J \geq 1$ одновременно начинается с a (как $w = a^J b \sigma(\Phi_1) \cdots$) и с b (как $w = b^I a \sigma(\Phi_2) \cdots$). Два слова, являющиеся префиксами общего слова w , сравнимы по префиксу; вычитая более короткий из них из более длинного, получаем уравнение того же вида для пары слов с меньшей суммарной длиной $|a| + |b|$ — шаг, повторяющий вычитательный вариант алгоритма Евклида для длин $|a|$ и $|b|$. Так как суммарная длина строго убывает, процесс конечен и завершается случаем, когда одно из слов пусто либо длины слов равны. На каждом шаге сохраняется наличие у w двух периодов, и по теореме 3 итоговый общий период $d = \gcd(|a|, |b|)$ задаёт корень z длины d , степенями которого являются a и b . Следовательно, $ab = ba$. $\square$

Лемма 7 является теоретической основой эвристики нормализации по классам коммутации: обнаружив в уравнении структуру (11), решатель может сразу заключить $xu = ux$ и избежать ветвления, порождаемого наивным нильсеновским разбиением.

2 Конструкторская часть

2.1 Функциональные требования

Постановка задачи — определение однозначности словарного образца — порождает следующий цикл обработки данных: образцы *вводятся, обрабатываются* каскадом методов возрастающей стоимости, результаты *сохраняются* в сводной базе и *выводятся* в удобном для пользователя виде. Требования к разрабатываемому программному обеспечению сведены в таблицу 1.

Таблица 1 — Функциональные и нефункциональные требования

№	Требование	Тип
Ф1	Приём образцов (строчные буквы — переменные, заглавные — константы) из CSV-файла или из командной строки	обяз.
Ф2	Сведение задачи однозначности к уравнению в словах (две копии переменных) и формирование SMT-LIB-задачи	обяз.
Ф3	Доказательство однозначности образца эвристикой балансирующих разрезов (лемма 5) без обращения к решателю	обяз.
Ф4	Доказательство неоднозначности каскадом критериев (унарный $\rightarrow$ плавные границы $\rightarrow$ бинарный) с обязательной проверкой свидетеля (леммы 3, 4)	обяз.
Ф5	Перенос установленного результата на родственные образцы по морфизмам (монотонность, лемма 2, следствие 1)	обяз.
Ф6	Запуск Ostrich (базовый и с классами коммутации) на нерешённых эвристиками образцах, разбор результата и контрпримера	обяз.
Ф7	Накопление в сводной базе записей вида (результат, источник, свидетель) и её использование как первого теста	обяз.
Ф8	Реализация в Ostrich общего алгоритма с классами коммутации (обнаружение, нормализация, прямое разрешение длин)	обяз.
Ф9	Вывод итогов в удобном виде: консоль и CSV (образец, результат, критерий, свидетель, слово-образ)	обяз.
НФ1	Корректность: каждый положительный результат сопровождается проверяемым свидетелем или леммой (отсутствие ложных срабатываний)	обяз.
НФ2	Инкрементальность: частичные результаты немедленно сбрасываются на диск	обяз.
НФ3	Параллельная обработка независимых образцов	жел.
НФ4	Интеграция в Ostrich без изменения публичного интерфейса теории строк (управление флагом сборки)	обяз.

2.2 Общая архитектура (каркас)

Система построена как каскад методов, упорядоченных по возрастанию вычислительной стоимости: дешёвые структурные критерии применяются первыми, а полный запуск решателя — последним, лишь для образцов, не разрешённых ни одним из предшествующих методов. Поток данных «ввод — обработка — хранение — вывод» показан на рисунке 1.

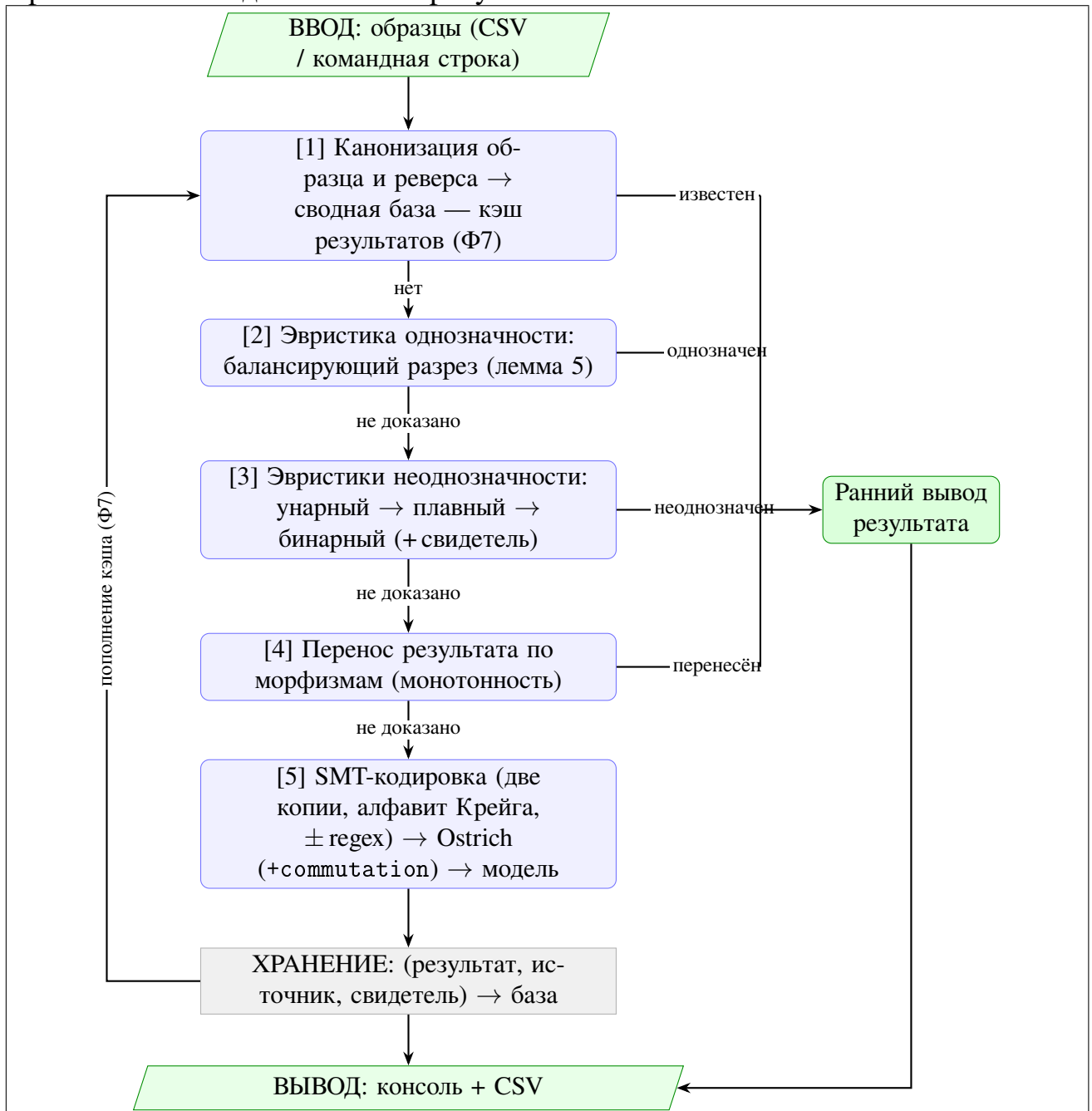


Рисунок 1 — Каркас системы: каскад методов возрастающей вычислительной стоимости и поток данных

Ключевое проектное решение — *первичная консультация с базой* (требование Ф7): прежде чем запускать какой-либо метод, обёртка проверяет, не известен ли результат для образца (или для родственного ему по морфизмам). Это превращает накопленную базу в кэш, монотонно ускоряющий повторные прогоны.

2.3 Проектирование интерфейса

Формат ввода. Образец — это слово в выбранной нотации: строчные латинские буквы ($a \dots z$) обозначают переменные (подставляются словами), заглавные ($A \dots Z$) — константы (буквы фиксированного алфавита). Допускаются две записи: посимвольная ($x A u y B u$) и через пробел ($x \ A \ u \ y \ B \ u$); многобуквенные литералы задаются в кавычках. Образцы подаются либо позиционными аргументами, либо в столбце `pattern` CSV-файла.

Формат вывода. Все инструменты выводят результат одновременно в консоль (для наблюдения за прогрессом) и в CSV (для дальнейшей обработки). Пользовательская строка результата содержит: образец, результат (ОДНОЗНАЧЕН / НЕОДНОЗНАЧЕН / не доказано), сработавший критерий или источник, свидетеля в читаемом виде $\sigma = \{ \dots \} \mid \tau = \{ \dots \}$ и слово-образ. Свидетель неоднозначности — это пара различных подстановок, дающих одно слово; свидетель однозначности — разрез $w_1 \mid w_2$ с переменной x_i и различающими константами $C_1 \neq C_2$.

2.4 Проектирование базы образцов

База образцов спроектирована как набор связанных таблиц (физически — CSV-файлы), центральная из которых сопоставляет каждому образцу результат и *источник* доказательства. Концептуальная схема записи приведена в таблице 2.

Физически база разложена по нескольким таблицам по стадиям обработки: входные образцы; результаты прогона решателя (статусы и время базовой и коммутационной конфигураций); проверенные результаты с верификацией контрпримера; корпус доказанных лемм-образцов (`pattern`, `verdict`, `flag`); образцы, выведенные переносом по морфизмам. Для *неоднозначного* образца источником служит контрпример (две подстановки) и указание происхождения — решатель Ostrich, суженный образец или эвристика; для *однозначного* — дерево доказательства (HTML + CSV) либо ссылка на релаксированный образец или лемму. Связи

Таблица 2 — Схема записи сводной базы образцов

Поле	Содержание
pattern	образец в канонической форме (ключ, см. 2.5)
verdict	ОДНОЗНАЧЕН / НЕОДНОЗНАЧЕН / НЕИЗВЕСТНО
source	источник результата: дерево доказательства (ссылка на HTML и CSV); контрпример (две подстановки); лемма или эвристика; ссылка на релаксированный или суженный образец
witness	свидетель: для неоднозначных — пара $\sigma \neq \tau$ с общим образом; для однозначных — балансирующий разрез
flag	пометка способа (лемма / эвристика / решатель / перенос)

«родительский образец $\rightarrow$ производный» реализуют перенос результатов из раздела 2.9.

2.5 Каноническая форма образца и ключ базы

Ключом сводной базы служит не сам образец, а его *каноническая форма*. Без нормализации образцы, отличающиеся лишь именами переменных и констант (например, $yxVxxAy$ и $baYaaxb$), считались бы разными и анализировались бы повторно. Приведение к каноническому виду устраняет эту избыточность: все образцы, являющиеся *переименованиями* (взаимно однозначной заменой переменных на переменные и констант на константы) одного и того же, получают общий ключ и находятся в базе сразу.

Каноническая форма строится двумя независимыми перенумерациями при одном проходе слева направо:

1. каждая *переменная* заменяется на её порядковый номер по первому вхождению: первая встреченная переменная — 1, вторая новая — 2, и так далее до n ;
2. каждая *константа* заменяется на букву из фиксированной последовательности $A, B, C, \dots, Z$ по порядку первого появления: первая новая константа — A , вторая — B и т. д.

Пример 6. Образец $yxVxxAy$ канонизируется так: переменные в порядке первого вхождения — $y \mapsto 1, x \mapsto 2$; константы — $V \mapsto A, A \mapsto B$. Получаем каноническую форму $1\ 2\ A\ 2\ 2\ B\ 1$. Любая его переименовка (например, $baYaaxb$) даёт ту же форму, поэтому повторного анализа не происходит.

В SMT-кодировке (подраздел 2.10) канонические номера переменных дают человеко-читаемые имена: σ -копия именуется $x_1, \dots, x_n$, τ -копия — $y_1, \dots, y_n$. Так, форме $1\ 2\ A\ 2\ 2\ B\ 1$ отвечает уравнение

$$x_1\ x_2\ A\ x_2\ x_2\ B\ x_1 = y_1\ y_2\ A\ y_2\ y_2\ B\ y_1.$$

Сверка с реверсом. Помимо канонизации самого образца, на первом шаге обёртка *реверсирует* образец, приводит реверс к канонической форме и также ищет его в базе. Это оправдано тем, что результат об однозначности инвариантен относительно разворота: образец $\mathcal{P}$ однозначен тогда и только тогда, когда однозначен его реверс $\mathcal{P}^R$ (разворот — биекция $\Sigma^* \rightarrow \Sigma^*$, сохраняющая конкатенацию с точностью до порядка, поэтому два различных свидетеля для $\mathcal{P}$ дают два различных свидетеля для $\mathcal{P}^R$, и наоборот). Поэтому если в базе уже есть канон реверса, результат переносится без повторного анализа; найденный для одного из двух образцов результат записывается под обоими ключами.

2.6 Обзор направлений проектирования

Содержательная часть проекта складывается из трёх направлений, каждое из которых реализует отдельный способ ответить на вопрос об однозначности образца и встроено в общий каркас (рисунок 1) как самостоятельный метод каскада.

1. *Структурные эвристики — обобщённый критерий Рейденбаха.* Дешёвые проверки, дающие строгий результат без обращения к решателю: эвристика однозначности по балансирующему разрезу (лемма 5), каскад критериев неоднозначности (унарный $\rightarrow$ плавные границы $\rightarrow$ бинарный, леммы 3, 4) и перенос результатов по морфизмам (лемма 2, следствие 1). Это направление обобщает критерий избыточности Рейденбаха (теорема 1) на образцы с константами и линейно зависимыми фрагментами (подразделы 2.7–2.9).
2. *SMT-спецификация задачи.* Кодирование задачи однозначности в формат SMT-LIB для нерешённых эвристиками образцов: выбор позитивной либо негативной формы, опора на явный алфавит констант (лемма 6) и, при необходимости, регулярные ограничения на алфавит переменных (подраздел 2.10).

3. *Доработка логики решателя.* Встраивание в Ostrich классов коммутации переменных, устраняющих избыточные ветвления нильсеновского разбиения на структуре леммы 7 (подраздел 2.11).

Ниже эти направления рассматриваются последовательно, после чего описывается объединяющий их сводный управляющий алгоритм.

2.7 Эвристика однозначности

Эвристика однозначности реализует лемму 5: образец однозначен, если существует балансирующий разрез, у которого обе части начинаются (или заканчиваются) одной переменной, за которой следуют различные константы. Балансирующие позиции вычисляются как разрезы с линейно зависимыми (пропорциональными) ненулевыми векторами кратностей частей. Алгоритм 1 перебирает балансирующие разрезы и проверяет префиксное и суффиксное условия леммы.

Листинг 1 Эвристика однозначности по балансирующему разрезу

```

1: function ДоказатьОднозначность( $\mathcal{P}$ )
2:   for  $i \in \text{БалансирующиеРазрезы}(\mathcal{P})$  do
3:      $w_1, w_2 \leftarrow \mathcal{P}[i], \mathcal{P}[i:]$ 
4:     for  $\text{усл} \in \{\text{Префикс}, \text{Суффикс}\}$  do
5:       if  $\text{усл}(w_1, w_2) = (x_i, C_1, C_2)$  и  $C_1 \neq C_2$  then
6:         return  $(|\text{vars}(\mathcal{P})| = 2 ? \text{ОДНОЗНАЧЕН} : \text{«однозначен по } x_i\text{»})$ , раз-
           рез  $w_1 \mid w_2$ 
7:   return не доказано

8: function Префикс( $w_1, w_2$ )
9:   return  $(w_1[0], w_1[1], w_2[1])$ , если  $w_1[0]=w_2[0]$  — переменная,  $w_1[1], w_2[1]$ 
   — константы; иначе  $\perp$ 

```

Здесь БалансирующиеРазрезы возвращает позиции i , для которых векторы $\nu(\mathcal{P}[i])$ и $\nu(\mathcal{P}[i:])$ линейно зависимы; проверка зависимости двух векторов сводится к обнулению всех миноров 2×2 .

2.8 Эвристики неоднозначности

Доказательство неоднозначности устроено как каскад из трёх критериев возрастающей общности; первый сработавший даёт ответ. *Любой* выданный свидетель проверяется подстановкой, поэтому положительный результат строг (требование НФ1).

1. *Унарный критерий* (лемма 3). Если есть буква-якорь M , кроме которой в образце лишь переменные и одна другая буква c , образец режется по вхождениям M на сегменты; для каждой переменной составляется вектор кратностей по сегментам. Нетривиальный целочисленный вектор ядра полученной матрицы задаёт два различных унарных присваивания над $\{c\}$ с общим образом.
2. *Критерий плавных границ* (лемма 4). Для периода $w = c_1c_2$ условие «значение каждой переменной — фактор w^∞ , а каждая константа стоит на своей фазе» записывается линейной системой над $\text{GF}(2)$ на чётностях длин переменных. При совместности системы перераспределение длины между двумя переменными (при неизменной суммарной длине образа) даёт $\sigma \neq \tau$ с равными образами.
3. *Бинарный критерий*. Конструктивный перебор присваиваний переменных короткими словами над алфавитом констант образца; коллизия образов (одно слово при разных присваиваниях) служит свидетелем. Глубина перебора выбирается адаптивно по размеру алфавита.

Листинг 2 Каскад доказательства неоднозначности

```
1: function ДоказатьНеоднозначность( $\mathcal{P}$ )
2:    $w \leftarrow$  Унарный( $\mathcal{P}$ )
3:   if  $w = \perp$  then  $w \leftarrow$  ПлавныеГраницы( $\mathcal{P}$ )
4:   if  $w = \perp$  then  $w \leftarrow$  Бинарный( $\mathcal{P}$ )
5:   if  $w = \perp$  then return не доказано
6:   if  $\hat{\sigma}_w(\mathcal{P}) = \hat{\tau}_w(\mathcal{P})$  и  $\sigma_w \neq \tau_w$  then
7:     return НЕОДНОЗНАЧЕН, свидетель  $w$ 
8:   return ошибка свидетеля
```

▷ не должно происходить (НФ1)

2.9 Перенос результатов по морфизмам

Монотонность однозначности и неоднозначности относительно морфизмов (лемма 2, следствие 1) позволяет переносить уже доказанный результат с образцами-леммы на целое семейство родственных образцов, не прибегая к решателю. Для целевого образца $\mathcal{Q}$ перебираются образцы-леммы $\mathcal{P}$ из корпуса и проверяется наличие одного из выводов:

- *образ по переменным*: $\varphi(\mathcal{P}) = \mathcal{Q}$, где φ заменяет переменные словами над переменными (сохранение однозначности);
- *прообраз по буквам*: $\psi(\mathcal{Q}) = \mathcal{P}$, где ψ — буквенный морфизм, не отождествляющий свидетелей (следствие 1); достаточный проверяемый случай — инъективное кодирование, распознаваемое алгоритмом Сардинаса–Паттерсона. Тест работает для нестирающих морфизмов, так как код не допускает пустых кодовых слов: например, на прообразе $xAuuyBxCx$ образца $xAuuyBxx$ (стирание $C \mapsto \varepsilon$) алгоритм Сардинаса–Паттерсона уже неприменим;
- *композиция* обоих переходов через промежуточный образец.

Найденный морфизм дополнительно *верифицируется* (он действительно переводит лемму в целевой образец), а корректность перехода по переменным гарантируется замыканием-сокращением блочных равенств (если в блоке не определена ровно одна переменная, её значения вынужденно совпадают). Свидетель неоднозначности переносится двойственным образом (прообраз по переменным, образ по буквам).

2.10 Спецификация SMT-задачи и выбор формы

Образцы, не разрешённые структурными эвристиками, передаются решателю. Для этого задача однозначности кодируется в формате SMT-LIB 2.6 (требование Ф2) по канонической форме образца (подраздел 2.5). Образец $\mathcal{P}$ с n переменными порождает две независимые копии: *левую* (σ -) копию с переменными $x_1, \dots, x_n$ и *правую* (τ -) копию с переменными $y_1, \dots, y_n$, где индекс — это канонический номер переменной. Равенство образов записывается строковым уравнением $\mathcal{P}[\sigma] = \mathcal{P}[\tau]$ через `str.++` (в обозначениях копий — x_i слева, y_i справа). Различие подстановок ($\sigma \neq \tau$) выражается отдельным условием, спосо-

бы записи которого рассматриваются ниже. Объявляемый алфавит сужается до явных констант образца по лемме 6. Конкретный вид порождаемого задания для образца $x A y x B y$ приведён в листинге 3.

Листинг 3: SMT-LIB-кодировка задачи однозначности образца $x A y x B y$ (канон 1 A 2 1 B 2, негативная форма)

```
(set-logic QF_S)
; левая (sigma) копия переменных: x1..xn по каноническим номерам
(declare-fun x1 () String) (declare-fun x2 () String)
; правая (tau) копия переменных: y1..yn
(declare-fun y1 () String) (declare-fun y2 () String)
; алфавит ограничен константами образца {A,B} (лемма Крейга)
(define-fun inAB ((w String)) Bool
  (str.in_re w (re.* (re.union (str.to_re "A") (str.to_re "B")))))
(assert (inAB x1)) (assert (inAB x2))
(assert (inAB y1)) (assert (inAB y2))
; равенство образов: x A y x B y для обеих копий (канон 1 A 2 1 B 2)
(assert (= (str.++ x1 "A" x2 x1 "B" x2)
  (str.++ y1 "A" y2 y1 "B" y2)))
; различие решений (негативная форма) -- через неравенство строк
(assert (or (distinct x1 y1) (distinct x2 y2)))
(check-sat)
(get-model)
```

Возможны две взаимодополняющие формы спецификации.

— *Негативная форма* (поиск свидетеля неоднозначности). Утверждается выполнимость конъюнкции

$$\mathcal{P}[\sigma] = \mathcal{P}[\tau] \wedge \bigvee_x (x^\sigma \neq x^\tau).$$

Результат `sat` означает *неоднозначность*, а возвращённая модель — готовый контрпример (две различные подстановки с общим образом); `unsat` означает *однозначность*. Эта форма используется по умолчанию, так как даёт проверяемый свидетель.

— *Позитивная форма* (доказательство однозначности). Однозначность есть общезначимость импликации $\mathcal{P}[\sigma] = \mathcal{P}[\tau] \Rightarrow \bigwedge_x x^\sigma = x^\tau$; её отрицание совпадает с негативной формой, поэтому позитивная форма решается проверкой невыполнимости. Она применяется, когда требу-

ется именно сертификат однозначности (дерево вывода `unsat`), а не контрпример.

Формулировка условия различия решений. Условие $\sigma \neq \tau$ нельзя записать одним атомом (кортежного неравенства в теории строк нет), поэтому используется дизъюнкция по переменным. Применяются две её формы:

- *строковое неравенство* (точная форма): $\bigvee_x (\text{distinct } x^\sigma x^\tau)$. Она *полна*: задание выполнимо тогда и только тогда, когда образец неоднозначен, поэтому `sat` и `unsat` дают окончательный результат.
- *сравнение длин*: $\bigvee_x (\text{str.len } x^\sigma \neq \text{str.len } x^\tau)$. В общем случае различие по длине — лишь достаточное условие различия слов, однако в данной постановке (обе стороны — две копии одного и того же канонического образца) эта форма *полна* и равносильна точной. Действительно, пусть уравнение $\mathcal{P}[\sigma] = \mathcal{P}[\tau]$ выполнено и при этом $|x_i^\sigma| = |x_i^\tau|$ для всех i . Тогда границы всех факторов на левой и правой сторонах совпадают (одинаковые длины у соответствующих переменных, константы общие), и, последовательно отщепляя ведущие переменные обеих сторон — равные по длине префиксы равных слов равны (лемма Леви, она же шаг нильсеновского отщепления), — получаем $x_i^\sigma = x_i^\tau$ для всех i , то есть $\sigma = \tau$. В противоположную сторону: при выполненном уравнении различие подстановок по значениям влечёт различие хотя бы по одной длине. Значит, длиновая и точная формы *равновыполнимы*, поэтому `sat` даёт свидетель неоднозначности, а `unsat` *доказывает* однозначность — запускать вторую, строковую, форму не требуется.

Длиновая форма дешевле и согласована со стратегией Ostrich, который сначала строит модель ограничений на длины: многие свидетели неоднозначности (в частности, унарные из леммы 3) как раз перераспределяют длину между переменными и улавливаются уже длиновым условием. В силу доказанной равновыполнимости обёртка использует именно её: и `sat`, и `unsat` здесь окончательны, так что отдельный проход с точным строковым неравенством не нужен и форма `distinct` остаётся лишь эталонной.

Выбор алфавита. По лемме 6 (достаточность явного алфавита) для спецификации без регулярных ограничений достаточно алфавита констант, явно входящих в образец: любую «постороннюю» букву модели можно заменить на одну

из этих констант, не нарушая ни уравнений, ни ограничений на длины. Поэтому в обеих формах объявляется конечный алфавит Σ_0 из встречающихся в образце заглавных букв (при их отсутствии — одна служебная константа). Сужение алфавита уменьшает пространство поиска и ускоряет нильсеновское разбиение Ostrich.

Регулярные ограничения. Если по условию значения переменных обязаны принадлежать заданному языку, в спецификацию добавляются ограничения `str.in_re`. Они фиксируют алфавит значений, из-за чего лемма 6 перестаёт применяться (заменять буквы уже нельзя), и, кроме того, влияют на стратегию разбиения: Ostrich сначала строит модель ограничений на длины, и регулярные условия, взаимодействуя с длиновыми, могут изменить выбор точки сплита (подраздел 1.13). Поэтому регулярные ограничения включаются только при необходимости и не отбрасываются «для скорости».

Выбор формы. По умолчанию обёртка формирует негативную форму с минимальным явным алфавитом: она быстрее и сразу даёт контрпример при неоднозначности. Позитивная форма с сертификатом запрашивается отдельно; регулярные ограничения добавляются лишь тогда, когда постановка явно ограничивает алфавит значений переменных.

2.11 Общий алгоритм с классами коммутации

Ядро доработки решателя — алгоритм, поддерживающий *классы коммутации* переменных: множества переменных, для которых уже доказано попарное коммутирование. Класс пополняется только при обнаружении структуры, гарантирующей коммутирование по лемме 7: уравнения вида $x^J y \Phi_1 = y^I x \Phi_2$ ($I, J \geq 1$, $x \neq y$ — нетерминалы). Классы хранятся структурой *система непересекающихся множеств* (Union-Find) с путевым сжатием; обнаружение описано алгоритмом 4.

Проверяется именно полная форма $x^J y \dots = y^I x \dots$, а не просто удвоенный префикс: объединение классов по неполному совпадению (например, лишь $a a \dots = b a \dots$) сливало бы переменные, не обязанные коммутировать, и приводило бы к некорректной нормализации подцелей.

Построенные классы используются на шаге разбиения уравнения двояко (алгоритм 5):

Листинг 4 Построение классов коммутации (Union-Find)

```
1: function КлассыКоммутации(уравнения-конкатенации цели)
2:    $parent \leftarrow \emptyset$  ▷ лес непересекающихся множеств
3:   for каждая пара конкатенаций  $s_1 = s_2$  с общим результатом do
4:     if Лемма-1( $s_1, s_2$ ) =  $(x, y)$  then Union( $x, y$ )
5:     if Лемма-1( $s_2, s_1$ ) =  $(x, y)$  then Union( $x, y$ )
6:     if  $s_1 = a b, s_2 = b a$  (бинарный прокси коммутации) then Union( $a, b$ )
7:   return классы размера  $\geq 2$ 

8: function Лемма-1( $s_1, s_2$ ) ▷ распознаёт  $x^J y \dots = y^I x \dots$ 
9:    $x \leftarrow s_1[0]; y \leftarrow s_2[0]$ 
10:  if  $x, y$  — нетерминалы и  $x \neq y$  then
11:     $J \leftarrow$  длина начального пробига  $x$  в  $s_1$ ;  $I \leftarrow$  длина начального пробига
     $y$  в  $s_2$ 
12:    if  $J \geq 1$  и  $I \geq 1$  и  $s_1[J] = y$  и  $s_2[I] = x$  then
13:      return  $(x, y)$ 
14:  return  $\perp$ 
```

- *нормализация без ветвления*: если обе ведущие переменные уравнения принадлежат одному классу, общий блок переменных класса сокращается с обоих концов, и уравнение переписывается в каноническую форму добавлением новых ограничений и удалением исходного — новые ветви доказательства не создаются;
- *прямое разрешение длин / коммутационное разбиение*: если коммутирование ещё не установлено, но обнаружена структура леммы 7, по арифметической модели длин проверяется делимость $|x|$ на $|y|$; при единственном допустимом значении кратности утверждается $x = y^k$ напрямую, иначе порождается специализированное разбиение, кодирующее $xy = yx$ и диофантово ограничение на длины.

Уравнения, для которых коммутативность уже закодирована (бинарный прокси $ab = t, ba = t$), исключаются из кандидатов на стандартное разбиение, что предотвращает заикливание.

Листинг 5 Нормализация либо разбиение с учётом классов коммутации

```

1: function РазбитьИлиНормализовать(уравнение  $E$ , классы  $C$ )
2:   if  $E$  — бинарный прокси коммутации then return  $\emptyset$            ▷ не трогаем
3:   if обе ведущие переменные  $E$  в одном классе  $C$  then
4:     сократить общий блок класса с обоих концов; добавить нормализован-
       ные ограничения, удалить  $E$ 
5:     return (без ветвления)
6:   else if в  $E$  обнаружена структура  $x^J y \dots = y^I x \dots$  then
7:     по модели длин: если  $|x| \bmod |y| = 0$  при единственной кратности  $k$ 
       to утвердить  $x = y^k$ 
8:     иначе породить разбиение с  $xy = yx$  и ограничением на длины
9:   else
10:    return СтандартноеНильсеновскоеРазбиение( $E$ )

```

2.12 Сводный управляющий алгоритм

Описанные компоненты объединяются обёрткой в единую процедуру принятия решения (алгоритм 6), реализующую каскад рисунка 1. Порядок шагов выбран так, чтобы дорогостоящий вызов решателя выполнялся только при неудаче всех дешёвых структурных методов, а накопленная база и корпус лемм сокращали повторную работу.

Листинг 6 Принятие решения об однозначности образца

```
1: function Решить(образец  $\mathcal{P}$ , база  $\mathcal{B}$ , корпус лемм  $\mathcal{L}$ )
2:    $\hat{\mathcal{P}} \leftarrow \text{Канон}(\mathcal{P})$ ;  $\hat{\mathcal{P}}^R \leftarrow \text{Канон}(\text{Реверс}(\mathcal{P}))$  ▷ нормализация (2.5)
3:   for  $k \in \{\hat{\mathcal{P}}, \hat{\mathcal{P}}^R\}$  do
4:     if  $k \in \mathcal{B}$  с определённым результатом then
5:       return  $\mathcal{B}[k]$  ▷ первый тест,  $\Phi_7$ ; реверс однозначен  $\iff$  образец
6:     далее работаем с каноном  $\hat{\mathcal{P}}$ , результат пишем под ключами  $\hat{\mathcal{P}}$  и  $\hat{\mathcal{P}}^R$ 
7:     if ДоказатьОднозначность( $\mathcal{P}$ ) успешна then
8:       записать в  $\mathcal{B}$ ; return ОДНОЗНАЧЕН
9:     if ДоказатьНеоднозначность( $\mathcal{P}$ ) успешна then
10:      записать (с контрпримером) в  $\mathcal{B}$ ; return НЕОДНОЗНАЧЕН
11:     if ПереносПоМорфизмам( $\mathcal{P}, \mathcal{L}$ ) успешен then
12:      записать в  $\mathcal{B}$ ; return перенесённый результат
13:      $S \leftarrow \text{КодироватьSMT}(\mathcal{P})$ ;  $r \leftarrow \text{Ostrich}(S, +\text{commutation})$ 
14:     результат  $\leftarrow (r = \text{unsat}: \text{ОДНОЗНАЧЕН}; r = \text{sat}: \text{проверить контрпример}$ 
        $\rightarrow \text{НЕОДНОЗНАЧЕН}; \text{иначе НЕИЗВЕСТНО})$ 
15:     записать (результат, источник, свидетель) в  $\mathcal{B}$ ; return результат
```

3 Технологическая часть

В разделе описаны технологические решения, отвечающие на вопрос «как реализовано»: стек технологий и внешние библиотеки, обвязка над решателем Ostrich, способы хранения, обхода и конвертации шаблонов, внутренние структуры Ostrich и встроенный в него вариант с классами коммутации, логирование и визуализация дерева вывода в HTML, а также описание программы и руководство пользователя.

3.1 Стек технологий и внешние библиотеки

Система состоит из двух слоёв: расширенного решателя Ostrich (Scala) и управляющей обвёртки (Python). Используемые технологии приведены в таблице 3.

Таблица 3 — Стек технологий и внешние библиотеки

Слой	Компонент	Назначение
Решатель	Scala 2.11.12	язык реализации решателя (кросс-сборка 2.12.20)
	sbt + sbt-assembly	система сборки, формирование единого jar
	Princess 2026-05-20	доказатель, предоставляет Goal, Plugin, Term, LinearCombination, ModelSearchProver
	SAT4J 2.3.1	SAT-ядро
	BRICS Automaton 1.11-8	автоматы для регулярных ограничений
	GraalVM native-image	опциональная сборка автономного исполняемого файла
Обвёртка	Python 3	язык управляющих скриптов и эвристик
	SymPy	вычисление целочисленного ядра матрицы (унарный критерий)
	стандартная библиотека	csv, subprocess, multiprocessing, itertools, json, argparse
	Graphviz (dot)	опциональный рендеринг дерева вывода в SVG/PNG
Форматы данных		SMT-LIB 2 (QF_SLIA) для задач; CSV для базы образцов; JSON для дерева вывода

3.2 Обязка над решателем Ostrich

Запуск решателя инкапсулирован в модуле `ostrich_runner`. Он находит самый свежий собранный `ostrich-assembly*.jar` в каталоге `target/scala-*`, формирует команду запуска JVM с увеличенным стеком и кучей (`-Xss40000k`, `-Xmx2000m`) и передаёт набор ключей решателя через параметр `-stringSolver`. Курсовой функционал включается *единственным* ключом `+commutation`; без него решатель ведёт себя как базовый Ostrich, что обеспечивает прямое сравнение конфигураций. Типичная команда запуска:

```
java -Xss40000k -Xmx2000m -cp ostrich-assembly.jar ostrich.OstrichMain \
  -stringSolver=ostrich.OstrichStringTheory:+nielsenSplitter,+commutation,-length=on \
  task.smt2
```

Ключ `-length=on` включает арифметическую модель длин, без которой не работают нильсеновское и коммутационное разбиение. Результат прогона возвращается структурой, содержащей статус (`sat/unsat/unknown`, извлекаемый как последнее вхождение в `stdout`), сам `stdout` с моделью, `stderr` с диагностическими логами (в том числе строки `[commutation]` и `[Nielsen-*)`), время работы и признак тайм-аута. Для каждого образца обвёртка запускает две конфигурации — базовую и с `+commutation` — и записывает статусы и время обеих.

3.3 Хранение, обход и конвертация шаблонов

Хранение. База образцов физически реализована набором CSV-файлов; образцы читаются из столбца `pattern`, результаты пишутся в выходной CSV. Запись ведётся *инкрементально* (требование НФ2): файл открыт на всё время прогона, каждая готовая строка немедленно записывается и сбрасывается на диск (`flush + fsync`); поскольку строка самодостаточна (содержит свой образец), прерывание прогона не теряет уже полученные результаты.

Параллелизм. Независимые образцы обрабатываются параллельно (требование НФ3): эвристики используют пул процессов (`multiprocessing.Pool.imap_unordered`), пакетный прогон решателя — пул потоков. Строки пишутся по мере готовности (в произвольном порядке), а по завершении переупорядочиваются по исходному номеру образца.

Обход. При разборе образца действует единое соглашение: строчная латинская буква — переменная (подставляется словом), заглавная — константа (буква фиксированного алфавита). Поддерживаются посимвольная запись ($xAyxBu$) и запись через пробел; многобуквенные литералы задаются в кавычках.

Конвертация данных. Преобразование образца в SMT-задачу выполняет модуль `gen_smt`: строятся две копии уравнения (переменные с суффиксами 1 и 2), различие копий выражается собственным суффиксом и ограничением на длины, а алфавит фиксируется регулярным ограничением `str.in_re`. Для образца $xAyxBu$ порождается задача вида:

```
(set-logic QF_SLIA)
(declare-fun x1 () String) (declare-fun y1 () String)
(declare-fun x2 () String) (declare-fun y2 () String)
(assert (str.suffixof x1 x2))
(assert (< (str.len x1) (str.len x2)))
(assert (str.in_re x1 (re.* (re.union (str.to_re "A") (str.to_re "B")))))
; ... аналогичные regex-ограничения для y1, x2, y2 ...
(assert (= (str.++ x1 "A" y1 x1 "B" y1)
           (str.++ x2 "A" y2 x2 "B" y2)))
(check-sat) (get-model)
```

Обратное преобразование — разбор результата решателя — выполняет модуль `check_uniqueness`: из контрпримера извлекаются две подстановки, которые подставляются в образец; результат НЕОДНОЗНАЧЕН принимается только при *проверенном* свидетеле (слова реально совпадают, а присваивания реально различны), что исключает ложные срабатывания.

3.4 Внутренние структуры Ostrich и вариант с классами коммутации

Механизм плагинов Princess. Теория строк Ostrich подключается к доказателю Princess через интерфейс плагина: метод `handleGoal(goal: Goal)` возвращает последовательность действий `Plugin.Action`, применяемых к состоянию доказательства. Обработка зависит от стадии цели — `Eager`, `Intermediate` или `Final`; действия сцепляются комбинатором `elseDo` (следующее применяется, только если предыдущее ничего не породило). Используются три типа действий:

- `Plugin.AddAxiom` — добавить ограничения в цель;
- `Plugin.RemoveFacts` — удалить обработанные факты (атомы конкатенации);
- `Plugin.AxiomSplit` — ветвление: цель делится на подцели, каждая со своей аксиомой и списком действий.

Представление термов. Строковые переменные и идентификаторы конкретных строк хранятся как `LinearCombination` (линейные комбинации над целыми); факты — предикатные атомы `Atom`, прежде всего `_str_++` (конкатенация) и `_str_len` (длина). Отображение `concatPerRes` сопоставляет каждому результирующему терму список конкатенационных литералов; наличие у одного результата двух определений и порождает уравнение, подлежащее разбиению. Конкретные значения длин вычисляются построением арифметической модели через `ModelSearchProver`.

Интеграция классов коммутации. Курсовой функционал управляется флагом `theoryFlags.commutation` (структура `OFlags`) и не меняет публичного интерфейса теории строк (требование НФ4). На стадии `Intermediate`, при включённых `+commutation` и `+nielsenSplitter`, перед стандартным нильсеновским разбиением выполняется коммутационное разбиение (`nielsenCommutationSplit`) с диагностическим выводом текущих классов; если оно неприменимо, управление передаётся стандартному `splitEquation`, который при включённой коммутации сначала пытается нормализацию без ветвления (`splitEquationWithLenNorm`).

Соответствие флагов и точек включения приведено в таблице 4.

Таблица 4 — Управляющие флаги и точки включения

Флаг	Действие
<code>+nielsenSplitter</code>	базовое нильсеновское разбиение уравнений
<code>+commutation</code>	классы коммутации: обнаружение, нормализация без ветвления, коммутационное разбиение (требует <code>+nielsenSplitter</code>)
<code>-length=on</code>	арифметическая модель длин (необходима обоим разбиениям)
<code>+backwardPropagation</code>	базовый профиль <code>Ostrich</code> (обратное распространение)

3.5 Логирование и визуализация дерева вывода

Логирование. Каждое решение нильсеновского разбиения (ветвление, нормализация, разложение) фиксируется логгером `DecisionTreeLogger` как узел JSON. Дерево перезаписывается после каждого добавления узла, причём запись выполняется во временный файл с последующим *атомарным* переименованием: даже при аварийном завершении решателя (например, по нехватке памяти) на диске остаётся последнее целостное дерево, а не усечённый файл. Путь к выходному файлу задаётся переменной окружения `OSTRICH_DECISION_TREE` или свойством `-Dostrich.decisionTree` (по умолчанию `decision_tree.json`).

Поскольку `Princess` исследует дерево доказательства в глубину, плоская последовательность решений недостаточна для восстановления ветвления. Логгер использует то, что множество ордуниг-констант цели монотонно растёт вниз по каждой ветви: *родителем* решения считается ранее залогированное решение, чьё множество констант — наибольшее подмножество текущего. Это позволяет восстановить реальную структуру родитель/ветвь и проиндексировать ветви среди потомков.

Визуализация. Модуль `visualize_decision_tree` читает JSON и для каждого узла строит переход «исходное уравнение $\rightarrow$ правило с параметрами $\rightarrow$ результирующие уравнения/ветви». Формируются три представления: цветной вывод в терминал; самодостаточный интерактивный HTML-файл `decision_tree.html` с раскрытием узлов и легендой правил (коммутационное разбиение, нормализация по классам, нильсеновское разбиение и разложения выделены разными цветами, рисунок 2); при наличии `dot` — граф потока вывода Graphviz в SVG/PNG, наглядно показывающий ветвление доказательства (рисунок 3).

3.6 Описание программы и руководство пользователя

Программа состоит из модуля-расширения решателя и набора управляющих скриптов обвёртки (таблица 5).

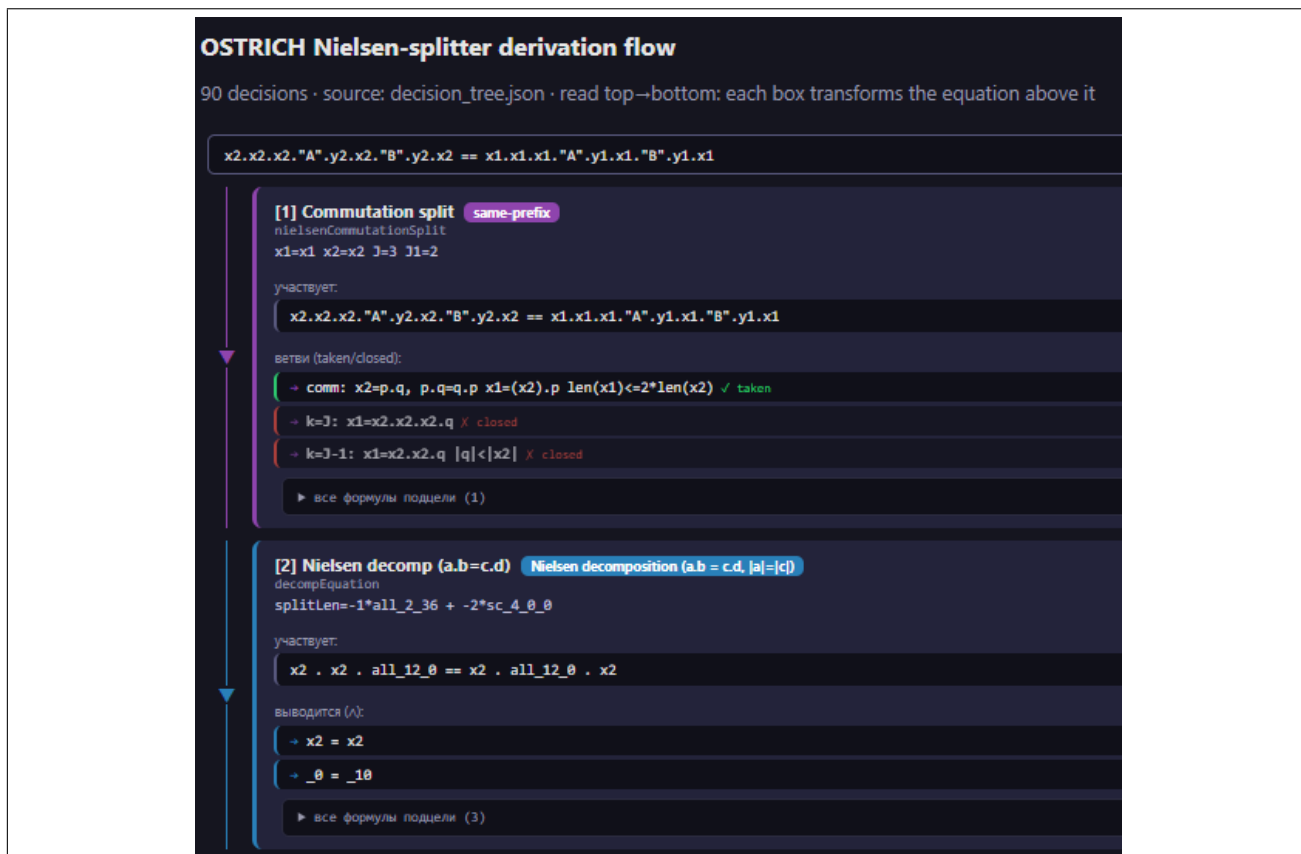


Рисунок 2 — Интерактивный HTML-вид дерева вывода

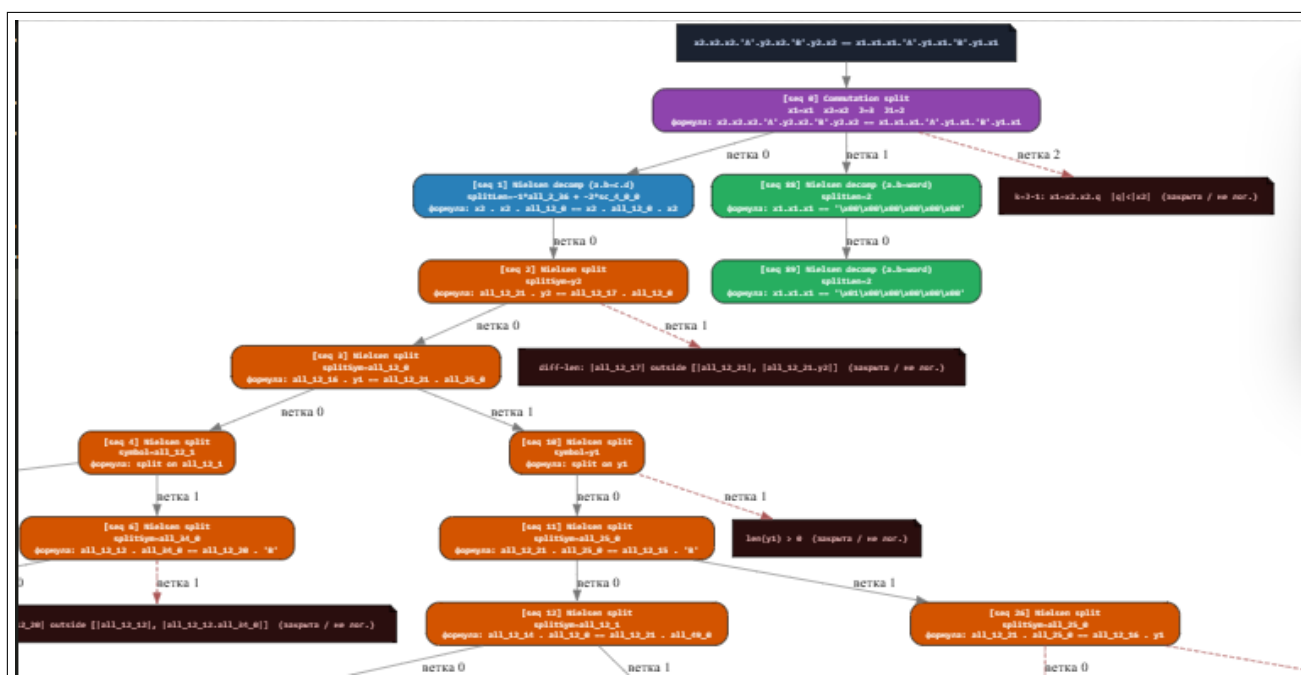


Рисунок 3 — Дерево разбора в виде графа

Требования. JDK 11 или выше, sbt, Scala 2.11; Python 3 с пакетом SymPy; опционально — Graphviz для рендеринга дерева вывода. Сборка решателя.

java -jar sbt-launch.jar assembly

Таблица 5 — Состав программных модулей

Модуль	Назначение
<code>OstrichNielsenSplitter.scala</code>	классы коммутации (Union-Find), обнаружение структуры леммы, нормализация и коммутационное разбиение, логгер дерева вывода
<code>OstrichStringTheory.scala</code>	точка интеграции: вызовы разбиения в <code>handleGoal</code>
<code>ostrich_runner.py</code>	запуск решателя, разбор статуса и логов
<code>gen_smt.py</code>	конвертация образца в SMT-LIB (две копии уравнения)
<code>run_patterns_csv.py</code>	пакетный прогон образцов (базовая и коммутационная конфигурации)
<code>ambiguity_heuristic.py</code>	эвристики неоднозначности (унарный, плавный, бинарный критерии)
<code>unambiguity_heuristic.py</code>	эвристика однозначности (балансирующие разрезы)
<code>prove_unambiguous_by_homomorphism.py</code>	перенос результатов по морфизмам
<code>check_uniqueness.py</code>	разбор контрпримера и верификация свидетеля
<code>visualize_decision_tree.py</code>	визуализация дерева вывода (HTML, Graphviz)

результат: `target/scala-2.11/ostrich-assembly*.jar`

Интерпретация вывода. В выходном CSV каждому образцу сопоставлены результат (ОДНОЗНАЧЕН / НЕОДНОЗНАЧЕН / не доказано), сработавший критерий или источник, свидетель ($\sigma \mid \tau$ для неоднозначности либо балансирующий разрез для однозначности) и слово-образ. Подробное дерево вывода решателя доступно в интерактивном файле `decision_tree.html`.

4 Аналитическая часть

4.1 Тестирование программного обеспечения и анализ результатов

В данном разделе представлен комплексный анализ производительности разработанного программного комплекса для определения (не)однозначности строковых шаблонов. Главной целью тестирования являлась оценка эффективности предложенного гибридного конвейера, включающего в себя алгоритм быстрого эвристического поиска и специализированный строковый SMT-решатель Ostrich (О-штрих) с внедренными архитектурными улучшениями. Универсальный решатель Z3 в рамках данного исследования применялся исключительно в качестве эталонного инструмента (baseline) для верификации результатов и создания базовой линии производительности при тестировании.

Ключевые показатели всех проведённых экспериментов сведены в таблицу 6; детальный разбор каждого этапа приведён в последующих подразделах. Таблица 6 — Сводные результаты экспериментального анализа методов конвейера

Метод / этап	Решено	Медиана	Примечание
Эвристики (быстрый фильтр), набор 118	91/118 (77%)	<1 мс	слово-свидетель; без ложных срабатываний
Z3 (эталон), набор 118	100/118 (85%)	1,94 с	выдаёт только статус sat/unsat
Ostrich, набор 118	98/118 (83%)	0,74 с	ускорение $\times 2,7$ к Z3; до $\times 1200$ на $xAxByCyx$
Ostrich + commutation, набор 118	117/118 (99%)	—	+2 задачи; пик $\times 11,3$; без регрессий
Полный конвейер, 221 нерешённый из базы 414	56 (25%)	—	30 лемм (однозн.) + 19 Ostrich + 7 эвристик (неодн.)

4.1.1 Верификация и оценка эвристического алгоритма

На первом этапе оценивалась работа быстрого эвристического алгоритма, основанного на поиске разрезов и балансировок. Для проверки корректности его работы и оценки полноты покрытия был сформирован тестовый набор из 118 уникальных строковых шаблонов. Результаты работы эвристики сверялись с выводами

эталонного тестировщика Z3, которому выделялся лимит времени в 200 секунд на каждый шаблон.

Эвристический подход продемонстрировал себя как высокоэффективный алгоритм первичной фильтрации, работающий практически мгновенно. Детальное сравнение производительности эвристического алгоритма и эталонного решателя на сформированной выборке показало следующие результаты. Эвристика успешно разрешила 91 задачу, что составило 77% от общего числа шаблонов, в то время как Z3 за выделенное время справился с 104 шаблонами (88%). При этом критическим отличием стало время выполнения: медианное время работы эвристического метода составило менее одной миллисекунды, тогда как эталонный решатель тратил на вычисления в среднем от 1 до 9 секунд. Оба инструмента продемонстрировали высокую надежность, не допустив ни одного ложного срабатывания (False Positive). Однако эвристический подход обладает неоспоримым преимуществом в части интерпретируемости, так как для каждого успешного решения он предоставляет конкретное слово-свидетель, в то время как классический решатель выдает исключительно статус выполнимости.

Сравнение с эталонным решателем полностью подтвердило корректность предложенного метода: тестовый инструмент подтвердил неоднозначность всех 91 шаблонов, решенных эвристикой, не выявив противоречий. Задачи, получившие статус «не доказано» (27 шаблонов), алгоритмически передавались на следующий этап конвейера — в специализированный решатель.

4.1.2 Оценка производительности специализированного решателя Ostrich

Для обработки сложных структурных шаблонов, не поддающихся быстрому эвристическому анализу, основным вычислительным ядром выступает строковый SMT-решатель Ostrich. Для подтверждения его эффективности было проведено нагрузочное тестирование в сравнении с базовым универсальным решателем на общей выборке из 118 шаблонов.

Тестирование показало подавляющее превосходство специализированного строкового решателя в скорости обхода дерева поиска. Сравнительная характеристика решателя Ostrich и эталонного инструмента выявила ряд фундаментальных различий в их поведении. Из 118 предложенных шаблонов базовый решатель Z3

успешно осилил 100 задач, тогда как Ostrich разрешил 98. Несмотря на незначительное отставание в абсолютном количестве решенных задач, медианное время выполнения на совместно решенных шаблонах у Ostrich составило всего 0.74 секунды против 1.94 секунды у эталонного инструмента, что обеспечивает среднее ускорение в 2.7 раза.

Анализ частоты превосходства по скорости показал, что Ostrich оказался быстрее в 75 случаях (63.5% выборки), в то время как Z3 опередил оппонента лишь 39 раз. Особого внимания заслуживают нетривиальные шаблоны, на которых специализированный алгоритм демонстрировал экстремальное ускорение: если максимальное зафиксированное ускорение для Z3 составило 17.5 раз, то Ostrich на задаче $x \wedge y \vee x \vee y$ сократил время поиска с 151 секунды до 0.12 секунды, показав превосходство в 1200 раз. Разработанный инструмент стабильно справлялся с пластом задач, вызывавших математический коллапс в универсальных алгоритмах, что визуально подтверждается логарифмическим распределением времени выполнения на соответствующем графике.

Важным аналитическим открытием стало выявление непересекающихся «слепых зон» у обоих алгоритмов. Было зафиксировано 16 уникальных решений, доступных только базовому инструменту, которые вызвали затруднения у Ostrich. В противовес этому, 14 шаблонов оказались разрешимы исключительно с помощью Ostrich, оказавшись недоступными для классической логики Z3 в рамках установленного лимита времени. Это обосновывает эффективность применения специализированных эвристик для узких классов задач.

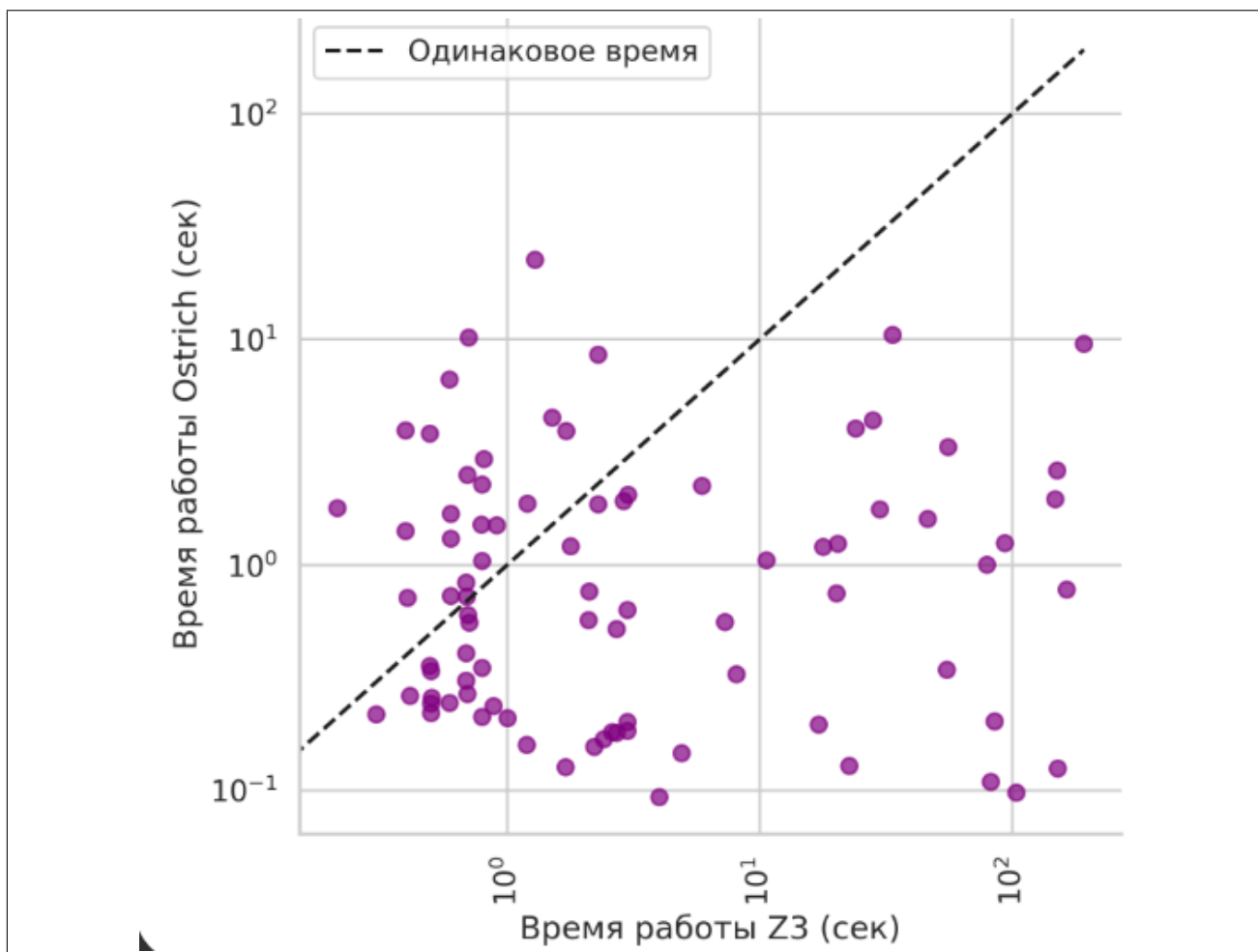


Рисунок 4 — Сравнение времени решения: Базовый инструмент против Ostrich (логарифмическая шкала)

4.1.3 Эффективность применения эвристики коммутативности

В рамках тонкой настройки решателя Ostrich было исследовано влияние эвристики коммутативности на производительность поиска. Тестирование проводилось с включенным и выключенным флагом коммутативности с целью выявления накладных расходов на дополнительные проверки симметрии.

Включение данной эвристики показало себя как алгоритмически безопасная оптимизация. Базовый режим без учета коммутативности позволил решить 116 задач, тогда как включение проверок симметрии расширило это количество до 117 успешно разрешенных шаблонов. Глобальное медианное время выполнения на всей тестовой выборке также продемонстрировало улучшение, снизившись с 505 секунды до 495 секунды. Детальный анализ характера влияния эвристики коммутативности на процесс поиска Ostrich выявил следующее распределение. В 56 случаях, что составляет около 47% от общего числа тестов, время работы ал-

горитма практически не изменилось. В 33 случаях (примерно 28%) наблюдалось существенное ускорение работы, достигающее в пиковых значениях превосходства в 11.3 раза за счет успешного отсека симметричных ветвей дерева решений. Естественное замедление из-за вычислительных затрат на дополнительные проверки было зафиксировано в 26 случаях (около 22%), однако степень этого замедления оказалась не критичной. Кроме того, именно применение эвристики позволило разрешить 2 ранее недоступные задачи (около 2% от выборки), которые при базовом прогоне уходили в таймаут. Итоговый вывод подтверждает необходимость интеграции данной эвристики в конвейер по умолчанию.

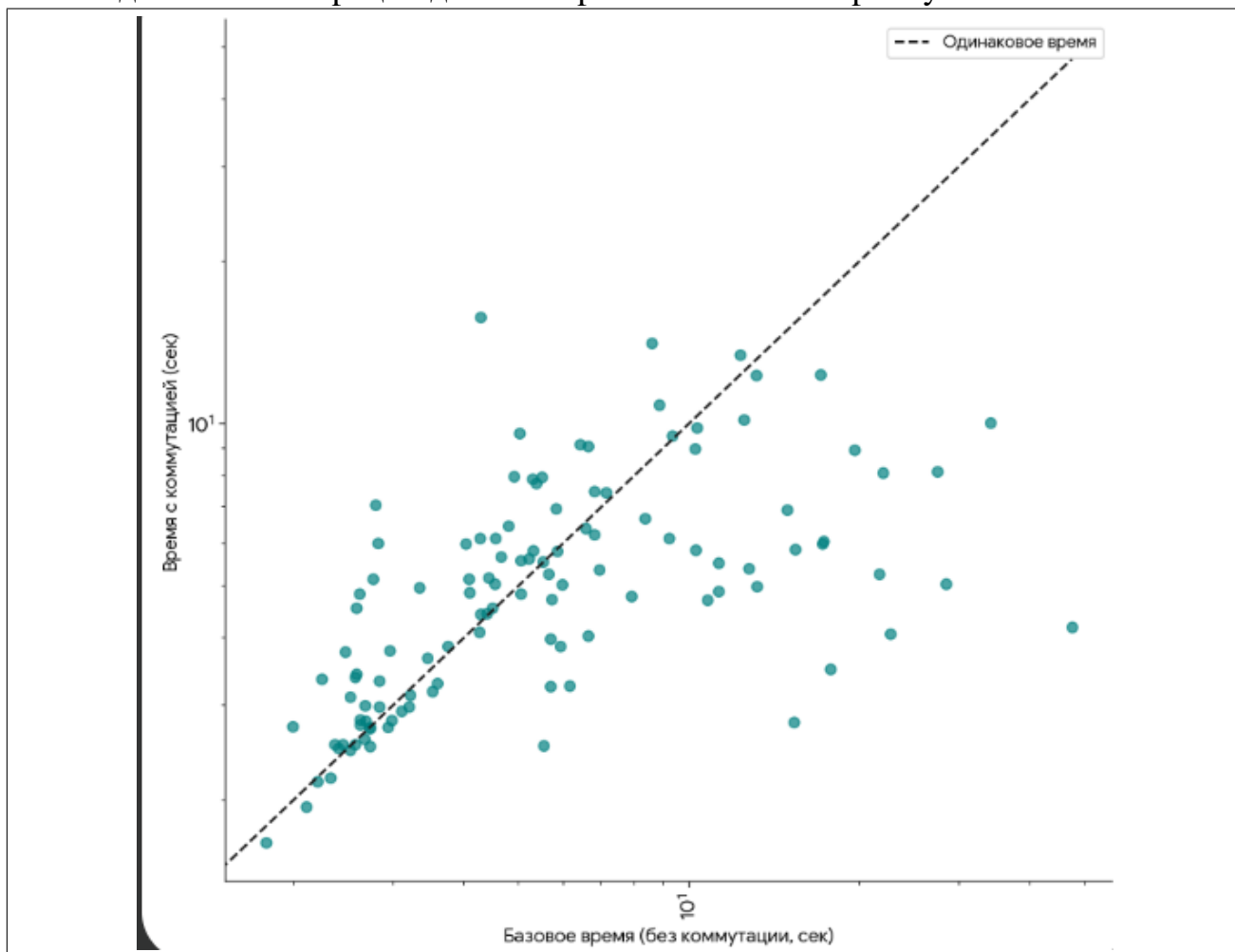


Рисунок 5 — Влияние эвристики коммутативности на время выполнения поиска

4.1.4 Анализ имеющейся базы данных и синтез методов

Финальным этапом аналитической работы стала проверка всего разработанного программного комплекса, включающего эвристический алгоритм, решатель Ostrich с настроенной эвристикой коммутативности и аппарат теоретических

лемм, на наиболее полной имеющейся базе данных. Исходная база содержала 414 шаблонов. Из них 179 были ранее классифицированы исследователями как неоднозначные, 14 — как однозначные, а подавляющее большинство, состоящее из 221 шаблона, имело статус строгой неопределенности и считалось алгоритмически неразрешенным.

Применение гибридного пайплайна позволило совершить существенный качественный скачок в покрытии пространства задач: из 221 ранее нерешенного шаблона конвейер смог выдать математически строгие доказательства для 56 новых случаев, что составляет 25% от числа всех зависших задач.

Структурная аналитика полученных решений демонстрирует четкое архитектурное разделение зон ответственности между компонентами предложенного конвейера. Математические леммы взяли на себя исключительно функцию доказательства однозначности и успешно обработали 30 тяжелых конфигураций. К этому классу относились сложные шаблоны длиной от 8 символов с тремя переменными и строгими перекрестными зависимостями (например, $xAxVuxSxu$ или $xAxyVuxSy$). Леммы оказались единственным инструментом, способным строго доказать однозначность таких структур.

В свою очередь, решатель Ostrich и эвристический алгоритм специализировались на поиске свидетелей неоднозначности. Решатель Ostrich взял на себя обработку нелинейных вычислительно сложных шаблонов, вызывавших комбинаторный взрыв при классическом переборе, и смог выявить 19 новых свидетелей неоднозначности. Эвристический алгоритм, опираясь на быстрые проверки балансировок, выявленные после оптимизации правил поиска, смог дополнительно доказать неоднозначность еще 7 шаблонов. В сумме конвейер обеспечил 56 новых доказательств (30 однозначных и 26 неоднозначных).

Валидация результатов на предмет регрессий подтвердила абсолютную надежность конвейера. Все 179 шаблонов, ранее доказанных как неоднозначные, и 14 однозначных шаблонов были верифицированы разработанным комплексом со стопроцентной точностью. Не было зафиксировано ни одного конфликтующего результата по отношению к имеющимся данным. Данная комплексная аналитика однозначно доказывает, что предложенная гибридная архитектура позволяет нивелировать аппаратные и алгоритмические ограничения изолированных SMT-

решателей, обеспечивая максимальную пробивную способность при анализе сложной строковой семантики.

ЗАКЛЮЧЕНИЕ

В работе изучены структурные свойства строковых образцов с повторными переменными и разработан программный комплекс для автоматического определения их однозначности. Комплекс включает теоретически обоснованные эвристики и расширение SMT-решателя Ostrich.

При разборе теоретических основ систематизированы известные алгоритмические результаты по уравнениям в словах.

Анализ проблемы бесконечного ветвления позволил выделить шаблон $x^J y \Phi_1 = y^I x \Phi_2$, на котором базовый алгоритм порождает потенциально бесконечное дерево вывода. На основе теоремы Файна–Вилфа и критерия коммутативности Линдона–Шютценберже доказана лемма о коммутировании переменных при удвоении; на неё опираются дальнейшие оптимизации.

Правило нормализации через классы коммутации реализовано в структуре Union-Find с путевым сжатием и встроено в модуль OstrichNielsenSplitter без изменения публичного интерфейса теории строк. Это позволило убрать избыточное ветвление: при включённой эвристике коммутации решатель разобрал на один шаблон больше, чем в базовом режиме; в 28% случаев получено ускорение до 11,3 раза, а медианное время на полной выборке снизилось с 5,05 до 4,95 секунды.

Дальнейшее развитие работы возможно в нескольких направлениях. Расширение алгоритма обнаружения коммутирующих классов на уравнения с тремя и более переменными охватит шаблоны, которые пока остаются нерешёнными. Перенос длиновых диофантовых ограничений прямо в шаг разбиения позволит отсекать часть ветвей ещё до их раскрытия. Наконец, автоматический вывод лемм-образцов из прогонов решателя и постепенное пополнение корпуса лемм могут ускорить обработку новых образцов за счёт накопленного знания.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Moura L. de, Bjørner N. Z3: An efficient SMT solver // Tools and Algorithms for the Construction and Analysis of Systems (TACAS). — Springer, 2008. — С. 337—340.
2. Decision Procedures for Path Feasibility of String-Manipulating Programs with Complex Operations / T. Chen [и др.] // Proceedings of the ACM on Programming Languages (POPL). Т. 6. — 2022. — С. 1—30.
3. Reidenbach D., Schneider J. C. Morphically primitive words // Theoretical Computer Science. — 2009. — Т. 410, № 21—23. — С. 2148—2161.
4. Freydenberger D. D., Reidenbach D., Schneider J. C. Unambiguous morphic images of strings // International Journal of Foundations of Computer Science. — 2006. — Т. 17, № 3. — С. 601—628.
5. Makanin G. S. The problem of solvability of equations in a free semigroup // Mathematics of the USSR-Sbornik. — 1977. — Т. 32, № 2. — С. 129—198.
6. Plandowski W. Satisfiability of word equations with constants is in PSPACE // Journal of the ACM. — 2004. — Т. 51, № 3. — С. 483—496.
7. Barrett C., Fontaine P., Tinelli C. The SMT-LIB Standard: Version 2.6 // Technical Report, The University of Iowa. — 2017.
8. Rümmer P. A Constraint Sequent Calculus for First-Order Logic with Linear Integer Arithmetic // Logic for Programming, Artificial Intelligence, and Reasoning (LPAR). — Springer, 2008. — С. 274—289.
9. Nielsen J. Die Isomorphismengruppe der freien Gruppen // Mathematische Annalen. Т. 91. — 1924. — С. 169—209.
10. Norn: An SMT Solver for String Constraints / P. A. Abdulla [и др.] // Computer Aided Verification (CAV). — Springer, 2015. — С. 462—469.
11. Lothaire M. Combinatorics on Words. — 2-е изд. — Cambridge University Press, 1997.
12. Fine N. J., Wilf H. S. Uniqueness theorems for periodic functions // Proceedings of the American Mathematical Society. — 1965. — Т. 16, № 1. — С. 109—114.

13. Lyndon R. C., Schützenberger M. P. The equation $a^M = b^N c^P$ in a free group // Michigan Mathematical Journal. — 1962. — T. 9, № 4. — C. 289—298.

ПРИЛОЖЕНИЕ А

Листинги программы

Ниже приведены листинги реализованных методов класса `OstrichNielsenSplitter`. Все изменения внесены в файл `src/main/scala/ostrich/proofops/OstrichNielsenSplitter.scala`.

Листинг 7: Разворачивание конкатенации в плоский список листовых термов

```
1 private def expandTerm(t: LinearCombination): Seq[LinearCombination] = {
2   if (strDatabase isConcrete t) {
3     List(t)
4   } else {
5     concatPerRes.get(t) match {
6       case Some(lits) if lits.size == 1 =>
7         expandTerm(lits.head(0)) ++ expandTerm(lits.head(1))
8       case _ =>
9         List(t)
10    }
11  }
12 }
13
14 // Any non-concrete term (user variable OR intermediate concat result)
15 private def isUngrounded(t: LinearCombination): Boolean =
16   !(strDatabase isConcrete t)
17
18 // Returns non-concrete terms that appear at least twice in the first 2
19 positions
20 private def doubledPrefix(seq: Seq[LinearCombination]):
21   Set[LinearCombination] = {
22     val pfx = seq.take(2)
23     pfx.filter(t => isUngrounded(t) && pfx.count(_ == t) >= 2).toSet
24   }
25
26 // Returns non-concrete terms that appear at least twice in the last 2
27 positions
28 private def doubledSuffix(seq: Seq[LinearCombination]):
29   Set[LinearCombination] = {
30     val sfx = seq.takeRight(2)
31     sfx.filter(t => isUngrounded(t) && sfx.count(_ == t) >= 2).toSet
32   }
33
34 private def prefixNonTerms(seq: Seq[LinearCombination]):
35   Set[LinearCombination] =
36     seq.take(2).filter(isUngrounded).toSet
37
38 private def suffixNonTerms(seq: Seq[LinearCombination]):
39   Set[LinearCombination] =
40     seq.takeRight(2).filter(isUngrounded).toSet
```

Листинг 8: Предикат isCommProxy и его использование в splitEquation

```
1 private def isCommProxy(lits: Seq[Atom]): Boolean =  
2   lits.size == 2 && {  
3     val a = lits(0); val b = lits(1)  
4     a(0) == b(1) && a(1) == b(0)  
5   }  
6  
7 def splitEquation : Seq[Plugin.Action] = {  
8   val multiGroups =  
9     concatPerRes filter {  
10      case (res, lits) =>  
11        lits.size >= 2 && !(strDatabase isConcrete res) &&  
12          !isCommProxy(lits)  
13      }  
14    // ... (остальная логика без изменений)  
15  }
```

Листинг 9: Метод findEquationsWithPrefixSuffixShared

```

1  def findEquationsWithPrefixSuffixShared()
2      : Seq[(Atom, Atom, LinearCombination,
3            Seq[LinearCombination], Seq[LinearCombination],
4            Set[LinearCombination], Set[LinearCombination])] = {
5      val result = new ArrayBuffer[...]
6
7      for ((res, lits) <- concatPerRes; if lits.size >= 2) {
8          for (i <- lits.indices; j <- (i + 1) until lits.size) {
9              val l1 = lits(i); val l2 = lits(j)
10             val lhs = expandTerm(l1(0)) ++ expandTerm(l1(1))
11             val rhs = expandTerm(l2(0)) ++ expandTerm(l2(1))
12
13             val lhsFirstIsNT = lhs.nonEmpty && isUngrounded(lhs.head)
14             val rhsFirstIsNT = rhs.nonEmpty && isUngrounded(rhs.head)
15
16             // a.a... == b.a... : doubled in one prefix, other side starts with
17             NT
18             val sharedPrefixSame =
19                 (if (rhsFirstIsNT) (doubledPrefix(lhs) intersect
20                                     prefixNonTerms(rhs))
21                  .filter(x1 => rhs.headOption.exists(_ !=
22                                     x1))
23                 else Set.empty[LinearCombination]) ++
24                 (if (lhsFirstIsNT) (doubledPrefix(rhs) intersect
25                                     prefixNonTerms(lhs))
26                  .filter(x1 => lhs.headOption.exists(_ !=
27                                     x1))
28                 else Set.empty[LinearCombination])
29
30             // x^I... == y^J... : both sides start with DIFFERENT doubled
31             non-terminals
32             val sharedPrefixCross =
33                 if (sharedPrefixSame.isEmpty &&
34                     lhsFirstIsNT && rhsFirstIsNT &&
35                     doubledPrefix(lhs).nonEmpty && doubledPrefix(rhs).nonEmpty)
36                     doubledPrefix(lhs)
37                 else Set.empty[LinearCombination]
38
39             val sharedPrefix = sharedPrefixSame ++ sharedPrefixCross
40             // ... аналогично для суффиксов ...
41
42             if (sharedPrefix.nonEmpty || sharedSuffix.nonEmpty)
43                 result += ((l1, l2, res, lhs, rhs, sharedPrefix, sharedSuffix))
44         }
45     }
46     result.toSeq
47 }

```

Листинг 10: Union-Find и метод computeCommutationClasses

```

1 private def ufFind(parent: MHashMap[LinearCombination,
2   LinearCombination],
3   x: LinearCombination): LinearCombination = {
4   if (!parent.contains(x)) parent(x) = x
5   if (parent(x) == x) x
6   else {
7     val root = ufFind(parent, parent(x))
8     parent(x) = root // нумеровое сжатие
9     root
10  }
11 }
12 private def ufUnion(parent: MHashMap[LinearCombination,
13   LinearCombination],
14   x: LinearCombination, y: LinearCombination): Unit = {
15   val rx = ufFind(parent, x); val ry = ufFind(parent, y)
16   if (rx != ry) parent(rx) = ry
17 }
18 def computeCommutationClasses(): Seq[Set[LinearCombination]] = {
19   val parent = new MHashMap[LinearCombination, LinearCombination]()
20
21   for ((res, lits) <- concatPerRes; if lits.size >= 2) {
22     for (i <- lits.indices; j <- (i + 1) until lits.size) {
23       val lhs = expandTerm(lits(i)(0)) ++ expandTerm(lits(i)(1))
24       val rhs = expandTerm(lits(j)(0)) ++ expandTerm(lits(j)(1))
25
26       // prefix: a.a... == b.a... => a commutes with b
27       if (rhs.nonEmpty && isUngrounded(rhs.head))
28         for (a <- doubledPrefix(lhs); if prefixNonTerms(rhs).contains(a))
29           ufUnion(parent, a, rhs.head)
30       if (lhs.nonEmpty && isUngrounded(lhs.head))
31         for (a <- doubledPrefix(rhs); if prefixNonTerms(lhs).contains(a))
32           ufUnion(parent, a, lhs.head)
33       if (lhs.size == 2 && rhs.size == 2 &&
34         isUngrounded(lhs.head) && isUngrounded(rhs.head) &&
35         lhs(0) == rhs(1) && lhs(1) == rhs(0))
36         ufUnion(parent, lhs(0), lhs(1))
37     }
38   }
39   val groups = new MHashMap[LinearCombination, Set[LinearCombination]]()
40   for (x <- parent.keys) {
41     val root = ufFind(parent, x)
42     groups(root) = groups.getOrElse(root, Set.empty) + x
43   }
44   groups.values.filter(_.size >= 2).toSeq
45 }

```

Листинг 11: Методы подсчёта листовых зон и построения нормализованных форм

```

1 private def prefixLenInClass(seq: Seq[LinearCombination],
2                               cls: Set[LinearCombination]): Int =
3   seq.indexWhere(!cls.contains(_)) match {
4     case -1 => seq.length
5     case n  => n
6   }
7
8 private def leafZoneCountsPrefix(seq: Seq[LinearCombination],
9                                   cls: Set[LinearCombination])
10    : Map[LinearCombination, Int] = {
11   val zone = seq.take(prefixLenInClass(seq, cls))
12   zone.groupBy(identity).map { case (k, v) => k -> v.size
13     }.withDefaultValue(0)
14 }
15
16 private def nonLeafTail(seq: Seq[LinearCombination],
17                           cls: Set[LinearCombination]):
18   Seq[LinearCombination] =
19   seq.drop(prefixLenInClass(seq, cls))
20
21 // Returns normalized forms for both sides: (P.RL.U', P.RR.V')
22 private def normalizedSeqBoth(doubledSide : Seq[LinearCombination],
23                               otherSide   : Seq[LinearCombination],
24                               cls         : Set[LinearCombination],
25                               isSuffix    : Boolean)
26    : (Seq[LinearCombination], Seq[LinearCombination]) = {
27   val nuL = if (isSuffix) leafZoneCountsSuffix(doubledSide, cls)
28             else          leafZoneCountsPrefix(doubledSide, cls)
29   val nuR = if (isSuffix) leafZoneCountsSuffix(otherSide, cls)
30             else          leafZoneCountsPrefix(otherSide, cls)
31   val mz   = cls.map(z => z -> math.min(nuL(z), nuR(z))).toMap
32   val sortedC = cls.toSeq.sortBy(z => (-mz(z), term2String(z)))
33   val P       = sortedC.flatMap(z => Seq.fill(mz(z))(z))
34   val RL      = sortedC.flatMap(z => Seq.fill(nuL(z) - mz(z))(z))
35   val RR      = sortedC.flatMap(z => Seq.fill(nuR(z) - mz(z))(z))
36   val uPr     = if (isSuffix) nonLeafHead(doubledSide, cls) else
37                 nonLeafTail(doubledSide, cls)
38   val vPr     = if (isSuffix) nonLeafHead(otherSide, cls) else
39                 nonLeafTail(otherSide, cls)
40   val normDoubled = if (isSuffix) uPr ++ RL ++ P else P ++ RL ++ uPr
41   val normOther   = if (isSuffix) vPr ++ RR ++ P else P ++ RR ++ vPr
42   (normDoubled, normOther)
43 }

```

Листинг 12: Нормализация уравнений при известных классах коммутации

```

1  def nielsenNormalize: Seq[Plugin.Action] = {
2    import TerForConvenience._
3    implicit val o = order
4
5    val eqs = findEquationsWithPrefixSuffixShared()
6    if (eqs.isEmpty) return List()
7
8    val knownClasses = computeCommutationClasses()
9    val elemToClass = (for (cls <- knownClasses; z <- cls) yield z ->
10      cls).toMap
11    val normActions = new ArrayBuffer[Plugin.Action]
12
13    // Priority: equations where x1, x2 already commute -> normalize
14    // without splitting
15    val alreadyKnown = eqs.flatMap { case (l1e, l2e, rese, lhse, rhse, pfx,
16      sfx) =>
17      val pfxPair = pfx.headOption.flatMap { x1e =>
18        val x2eOpt = if (doubledPrefix(lhse).contains(x1e)) rhse.headOption
19          else lhse.headOption
20        x2eOpt.filter(x2e => elemToClass.get(x1e).exists(_.contains(x2e)))
21          .map(x2e => (l1e, l2e, rese, lhse, rhse, x1e, x2e, false))
22      }
23      pfxPair.orElse(/* suffix analogue */ None)
24    }
25
26    if (alreadyKnown.nonEmpty) {
27      for ((l1e, l2e, rese, lhse, rhse, x1e, x2e, isSfx) <- alreadyKnown) {
28        val (normD, normO) = normalizedSeqBoth(lhse, rhse, Set(x1e, x2e),
29          isSfx)
30        if (normD != lhse || normO != rhse) {
31          val normFormula = {
32            val builder = new FormulaBuilder(goal, theory)
33            if (normD.nonEmpty) builder.addConcatN(normD, rese)
34            if (normO.nonEmpty) builder.addConcatN(normO, rese)
35            builder.result
36          }
37          normActions += Plugin.AddAxiom(concatLits ++ lengthLits,
38            normFormula, theory)
39          normActions += Plugin.RemoveFacts(Conjunction.conj(List(l1e,
40            l2e), order))
41        }
42      }
43    }
44    return normActions.toSeq
45  }
46  normActions.toSeq
47 }

```

Листинг 13: Трёхветвевое разбиение по коммутации

```

1  def nielsenCommutationSplit: Seq[Plugin.Action] = {
2      import TerForConvenience._
3      implicit val o = order
4
5      val eqs = findEquationsWithPrefixSuffixShared()
6      if (eqs.isEmpty) return List()
7
8      val (l1, l2, res, lhs, rhs, sharedPfx, sharedSfx) = eqs.head
9      val actions = new ArrayBuffer[Plugin.Action]
10
11     def doSplit(x1: LinearCombination, x2: LinearCombination,
12                doubledSide: Seq[LinearCombination],
13                otherSide: Seq[LinearCombination],
14                atomToRemove: Atom, isSuffix: Boolean): Unit = {
15
16         val J = if (isSuffix) otherSide.reverseIterator.takeWhile(_ == x2).length
17                 else          otherSide.iterator.takeWhile(_ == x2).length
18         val J1 = math.max(1, J - 1)
19         val commFormula = {
20             val builder = new FormulaBuilder(goal, theory)
21             val t = builder.newVar(StringSort)
22             builder.addConjunct(_str_++(List(l(x1), l(x2), l(t))))
23             builder.addConjunct(_str_++(List(l(x2), l(x1), l(t))))
24             if (J1 > 0)
25                 for (lenX1 <- lengthMap.get(x1); lenX2 <- lengthMap.get(x2))
26                     builder.addConjunct(
27                         sum(List((IdealInt(J1), l(lenX2)), (IdealInt.MINUS_ONE, l(lenX1)))) >= 0)
28             builder.result
29         }
30         val caseJFormula = {
31             val builder = new FormulaBuilder(goal, theory)
32             val q = builder.newVar(StringSort)
33             if (isSuffix) builder.addConcatN(q +: Seq.fill(J)(x2 : Term), x1)
34             else          builder.addConcatN(Seq.fill(J)(x2 : Term) :+ q, x1)
35             builder.result
36         }
37         val caseJ1Formula = {
38             val builder = new FormulaBuilder(goal, theory)
39             val q = builder.newVar(StringSort)
40             if (isSuffix) builder.addConcatN(q +: Seq.fill(J1)(x2 : Term), x1)
41             else          builder.addConcatN(Seq.fill(J1)(x2 : Term) :+ q, x1)
42             builder.result
43         }
44         val doubledAtom: Atom = if (atomToRemove eq l2) l1 else l2
45         val removeBoth = Conjunction.conj(List(atomToRemove, doubledAtom), order)
46         val removeOther = Conjunction.conj(atomToRemove, order)
47         actions += Plugin.AxiomSplit(
48             concatLits ++ lengthLits,
49             List(
50                 (commFormula, List(Plugin.RemoveFacts(removeBoth))),
51                 (caseJFormula, List(Plugin.RemoveFacts(removeOther))),
52                 (caseJ1Formula, List(Plugin.RemoveFacts(removeOther)))
53             ),
54             theory
55         )
56     }
57 }

```